



RAPPORT DE PROJET

CoVACIEL 2026

Voiture Autonome pour la Compétition CIEL

Jonathan JAEGER • Quentin STUTZMANN • Hugo KRAWIEC • Betul ARTI

BTS CIEL2 — Lycée Heinrich Nessel, Haguenau

Compétition CoVACIEL — Juin 2026

Encadrant électronique : Emmanuel KEITH

Site officiel : www.covaciel.fr

Sommaire

Table des matières

Sommaire.....	2
1. Cahier des charges	5
1.1 Contexte et objectifs	5
1.2 Contraintes réglementaires	5
1.3 Exigences fonctionnelles.....	6
2. Plannings	7
2.1 Planning prévisionnel.....	7
2.2 Planning de suivi.....	7
3. Architecture système	8
3.1 Diagramme de bloc.....	8
3.2 Diagramme de cas d'utilisation.....	10
3.3 Diagramme d'exigences.....	10
4. Parties communes.....	11
4.1 Répartition des tâches	11
4.2 Cartes électroniques	11
4.3 Choix du microcontrôleur	11
5. Partie Avant — Quentin STUTZMANN	12
5.1 Intégration LiDAR.....	12
Caractéristiques et principe de fonctionnement	12
Prise en main avec le logiciel Frame Grabber	12
Programmes de test Python — LidarTest910 et LidarTest913.....	13
Difficultés rencontrées — Buffer série saturé.....	15
Structure de la grille 7×3	15
Bug critique résolu — Décodage des angles	16
Extraits de code clés	16
Difficultés rencontrées et solutions adoptées.....	17
5.2 Caméra HuskyLens	17
Caractéristiques et principe de fonctionnement	17
Stratégie de traitement et protocole I2C direct.....	18
Extraits de code clés	18
Difficultés rencontrées et solutions adoptées.....	20
6. Partie arrière — Hugo KRAWIEC	21
6.1 Réalisations	21
6.1.1 Capteur ultrason arrière — SRF10	21
Caractéristiques techniques	21
Tableau comparatif SRF10 vs HC-SR04	21

Intégration sur l'ESP32	22
Logique de sécurité intégrée	22
Code de pilotage du SRF10	22
6.1.2 Module XBee Pro — Signal de départ	23
Configuration radio	23
Trames de contrôle	23
6.1.3 Buzzer — Signalisation sonore.....	23
6.2 Difficultés rencontrées et solutions adoptées	24
6.2.1 Choix du capteur arrière	24
6.2.2 Durée de mesure et contrainte temps-réel.....	24
6.2.3 Migration STM32 → ESP32.....	24
6.2.4 Configuration du réseau XBee.....	24
7. Pilotage autonome — Jonathan JAEGER	25
7.1 Présentation du système côté Raspberry Pi.....	25
7.2 Réception et traitement des données JSON	25
7.3 Algorithme de décision RACE_CENTER.....	26
Simulation préalable et sélection du profil	26
Règles de décision	26
Rampe d'accélération et de freinage	28
Marche arrière.....	28
7.4 Commande des actionneurs via pigpio	28
7.5 Alimentation du servomoteur — problème et solution	29
Problème rencontré lors des tests sur piste.....	29
Diagnostic	29
Solution : alimentation par le BEC de l'ESC	29
7.6 Interface de monitoring temps réel.....	30
7.7 Surveillance automatique des communications.....	31
7.8 Méthodologie de test et validation.....	31
Tests unitaires de l'algorithme (200 grilles aléatoires)	31
Tests sur piste par étapes successives	32
7.9 Optimisations réalisées côté Raspberry Pi.....	32
7.10 Historique des versions.....	33
8. Dashboard Web — Betul ARTI	34
8.1 Rôle et objectifs du site	34
8.2 Technologies utilisées.....	35
8.3 Conception de la base de données.....	35
Modèle logique des données (MLD).....	35
Table utilisateurs	36
Table courses.....	36

Table télémetrie.....	37
Migration PC → Raspberry Pi.....	37
8.4 Diagrammes de conception	38
Diagramme de séquence	38
Diagramme de cas d'utilisation	39
8.5 Réalisation du site web	40
Structure et navigation.....	40
Dashboard temps réel — WebSocket / Socket.IO	40
Page statistiques	41
8.6 Hébergement sur Raspberry Pi 400	42
Environnement virtuel Python (venv).....	42
Installation de la pile LAMP	42
8.7 Configuration réseau et sécurisation.....	43
Création du hotspot Wi-Fi.....	43
Sécurisation par pare-feu (UFW).....	44
Protocole WPA3	44
Sécurité applicative	44
Procédure de lancement	45
8.8 Extraits de code clés.....	45
Lancement du serveur Flask	45
Gardien de sécurité — Vérification de session	45
Hachage SHA-256 et authentification.....	45
Push WebSocket temps réel	46
Route statistiques.....	47
8.9 Problèmes rencontrés et solutions	47
8.10 Bilan personnel	47
9. Bilan et conclusion.....	48
9.1 Bilan des réalisations	48
9.2 Leçons clés du projet	49
9.3 Conclusion	49
Index des figures	50

1. Cahier des charges

1.1 Contexte et objectifs

CoVACIEL (Voiture Autonome pour la Compétition CIEL) est une compétition nationale organisée entre établissements BTS CIEL. Chaque équipe transforme une voiture de modélisme en véhicule entièrement autonome, capable de naviguer sur un circuit balisé avec obstacles sans aucune intervention humaine.

Notre équipe du Lycée Heinrich Nessel de Haguenau (BTS CIEL2, promotion 2025–2026) est composée de quatre étudiants :

Étudiant	Domaine	Responsabilités principales
Jonathan JAEGER	Pilotage	Traitement LiDAR, algorithme RACE_CENTER, Raspberry Pi, commande servo et moteur, intégration système
Quentin STUTZMANN	Partie Avant	Câblage complet, intégration mécanique sur le châssis, tests de continuité
Hugo KRAWIEC	Partie Arrière	Capteur ultrason SRF10 (I2C), module XBee signal de départ, buzzer, sécurité marche arrière
Betul ARTI	Dashboard Web	Serveur Flask, base SQLite, WebSocket, interface Chart.js, sécurisation réseau

1.2 Contraintes réglementaires

Contrainte	Valeur	Impact sur le projet
Budget	150 € maximum	Composants accessibles : ESP32, batterie NiMH
Durée	18 semaines (jan. à juin 2026)	Planning serré, méthode itérative
Revue	3 revues obligatoires	Documentation intermédiaire requise
Course	3 minutes	Optimisation conjointe vitesse et fiabilité
Pilotage	100 % autonome	Aucune intervention après le signal de départ
Départ	XBee sans fil	Synchronisation avec le jury depuis la ligne de départ
Obstacles	Présents sur le circuit	Détection et évitement obligatoires en temps réel
Compétition	Juin 2026, Haguenau	Intégration finalisée avant mai 2026

1.3 Exigences fonctionnelles

Les exigences sont organisées selon la hiérarchie SysML de la section 3.3 :

Déplacement	Communication et sécurité
- Id 1.1 — Détecter l'environnement (RPLIDAR A2M12 avant, SRF10 arrière) - Id 1.2 — Traiter les données et décider la trajectoire (ESP32 et Raspberry Pi) - Id 1.3.1 — Contrôler la propulsion : accélération progressive, freinage adaptatif - Id 1.3.2 — Contrôler la direction : braquage entre -45° et +45° - Id 1.3.3 — Marche arrière : vérification de sécurité puis exécution	- Id 1.4.1 — Recevoir le signal XBee et activer le mode automatique - Id 1.4.2 — Signaler l'état du véhicule (2 bips buzzer, LED) - Id 1.4.3 — Transmettre les données vers le PC de supervision - Id 1.4.4 — Afficher le tableau de bord web en temps réel - Id 1.5.2 — Latence ≤ 5 ms, fréquence ≥ 50 Hz, arrêt auto ≤ 500 ms

2. Plannings

2.1 Planning prévisionnel

Établi en janvier 2026, le planning couvre les 18 semaines de réalisation réparties entre cinq domaines : FRONT (partie avant électronique), BACK (capteurs arrière), TRACTION et PILOTAGE, WEB (dashboard) et activités transverses. Le nom du responsable est précisé pour chaque tâche.

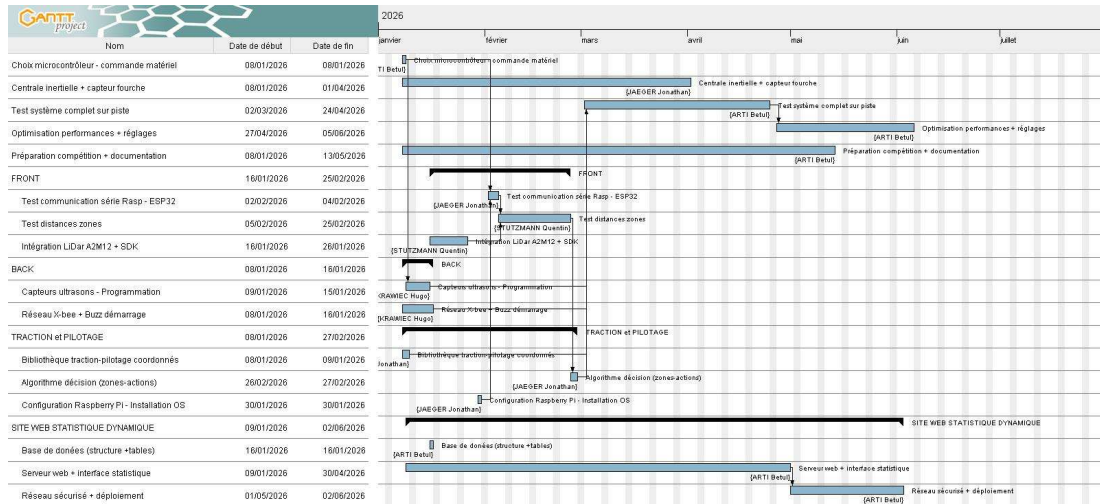


Figure 1 - Planning prévisionnel du projet CoVACIEL (janvier à juin 2026)

2.2 Planning de suivi

Le planning de suivi retrace l'avancement réel semaine par semaine et met en évidence les écarts par rapport au prévisionnel.

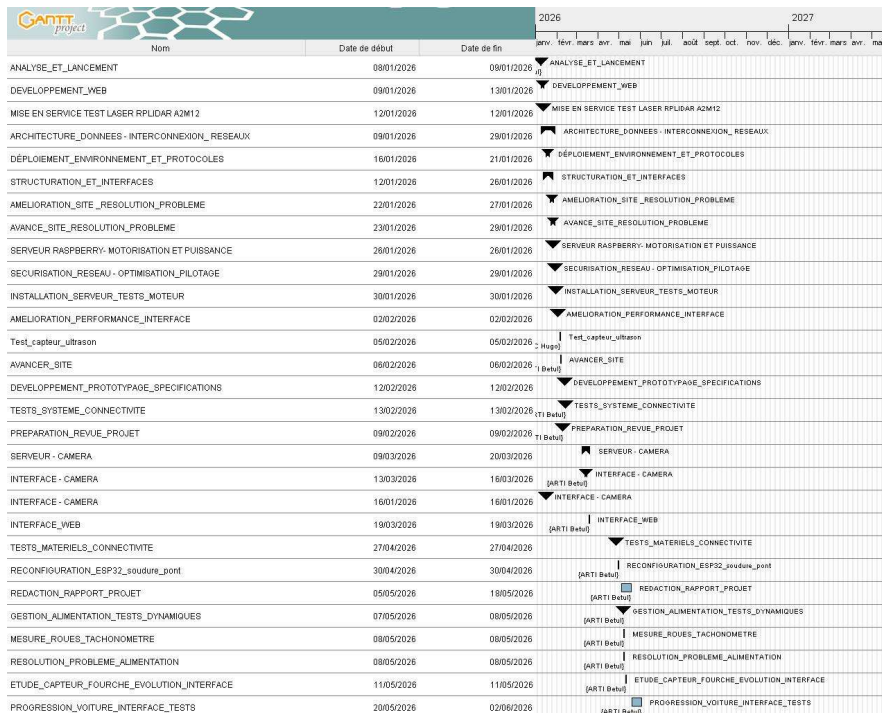


Figure 2 - Planning de suivi hebdomadaire du projet CoVACIEL (janvier à mai 2026)

3. Architecture système

3.1 Diagramme de bloc

Le diagramme de bloc SysML décompose le système en six sous-systèmes : Capteurs, Traitement, Actionneurs, Interface HMI, Châssis Mécanique et Stockage Énergie.

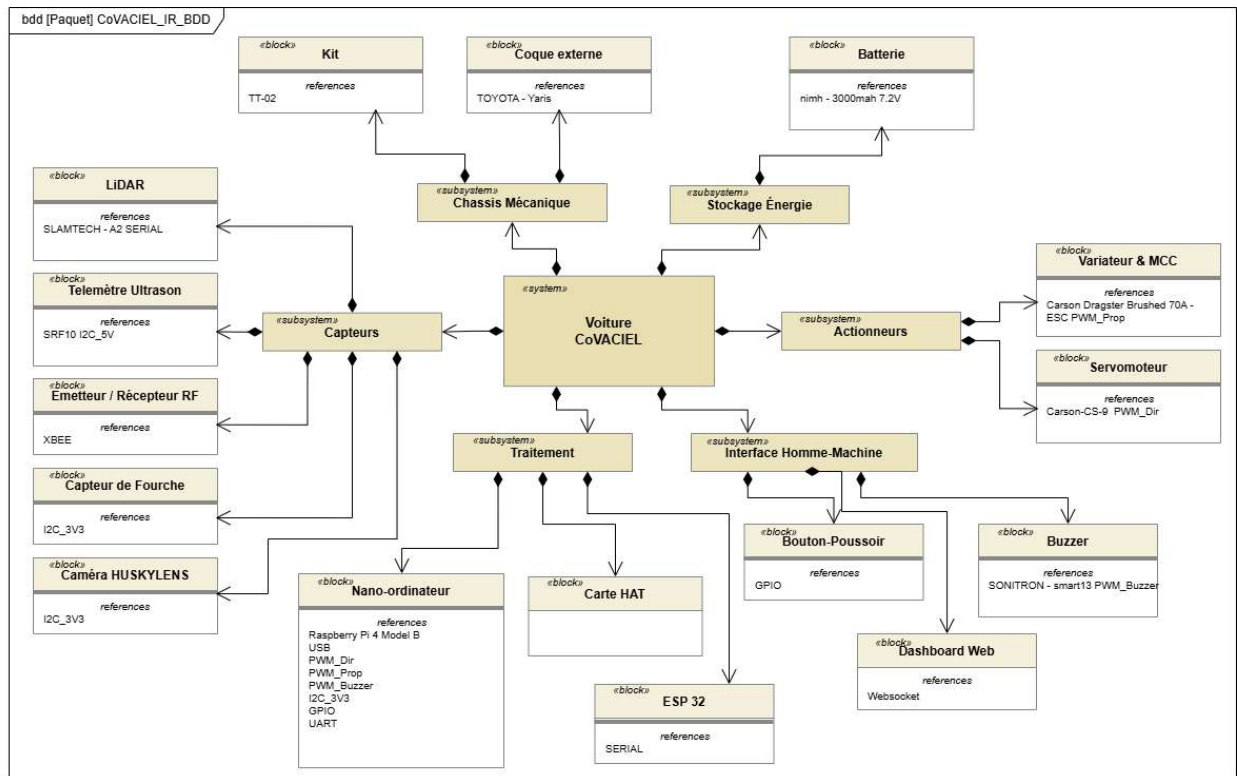


Figure 3 - Diagramme de bloc SysML — Architecture matérielle du système CoVACIEL

- Traitement : Raspberry Pi 4 Model B (décisions de pilotage) et ESP32 WROOM (acquisition LiDAR)
- Capteurs : RPLIDAR A2M12 (UART), SRF10 ultrason (I2C 5V), XBee radiofréquence, fourche optique (I2C 3,3V), HuskyLens (I2C 3,3V)
- Actionneurs : variateur ESC TAMIYA (PWM propulsion) et servomoteur Carson CS-9 (PWM direction)
- Interface HMI : bouton-poussoir (GPIO) et buzzer SONITRON smart13
- Châssis : kit TT-02, carrosserie Toyota Yaris, Énergie : batterie NiMH 3000 mAh 7,2 V

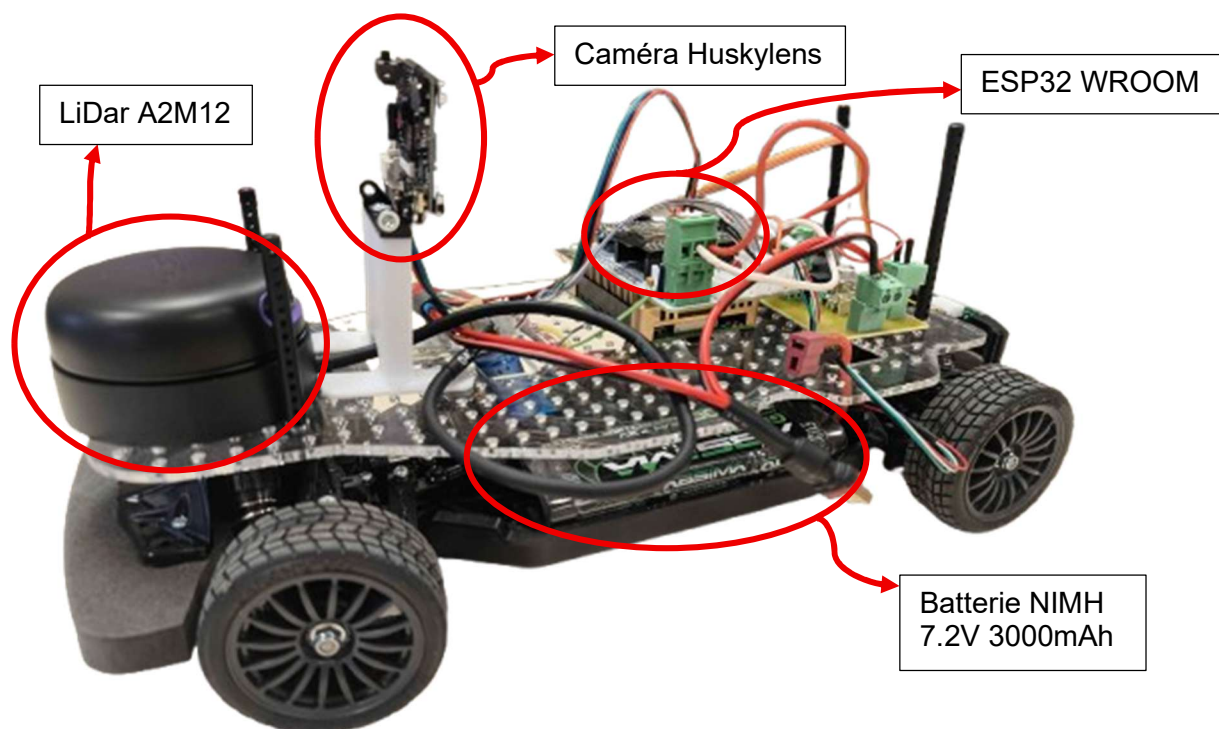


Figure 4 – Photo voiture CoVACIEL avant 3/4

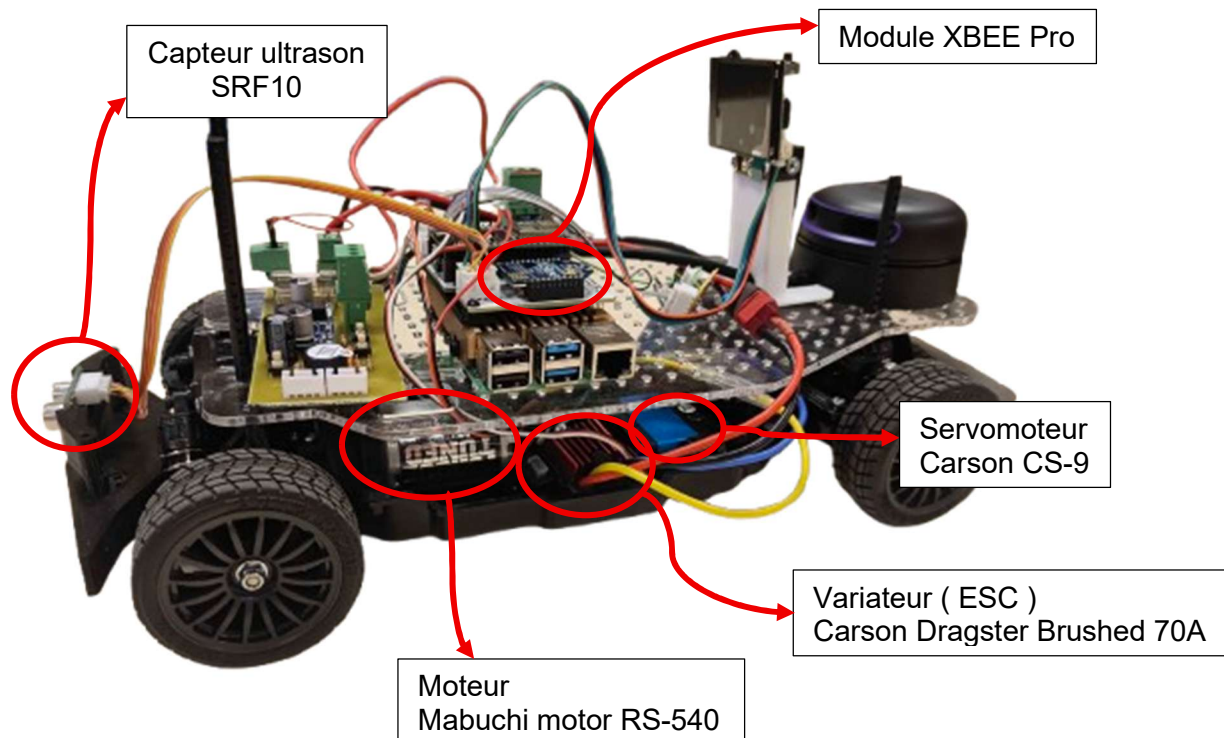


Figure 5 – Photo voiture CoVACIEL arrière 3/4

3.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation identifie les interactions avec trois acteurs : la Voiture (système embarqué), le Superviseur de course et le PC de Monitoring.

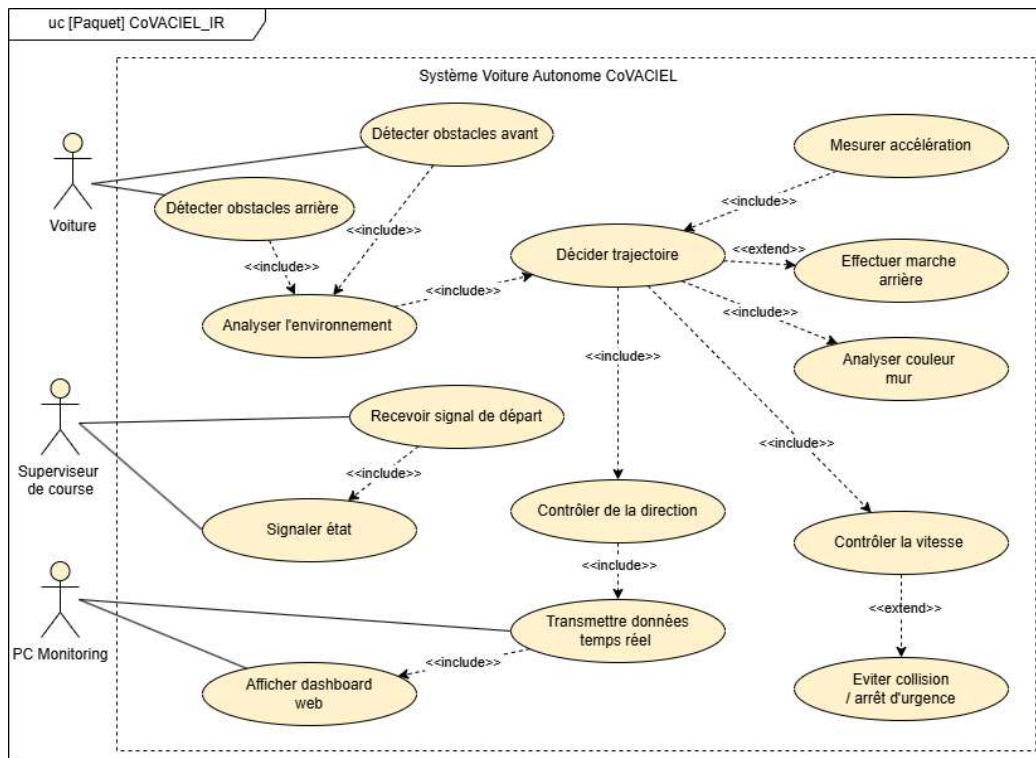


Figure 6 - Diagramme de cas d'utilisation SysML — CoVACIEL

3.3 Diagramme d'exigences

Le diagramme d'exigences organise hiérarchiquement les 15 exigences du système. L'exigence racine (Id=1) est décomposée via des liens $\oplus$, puis affinée via <<refine>> pour les précisions quantitatives.

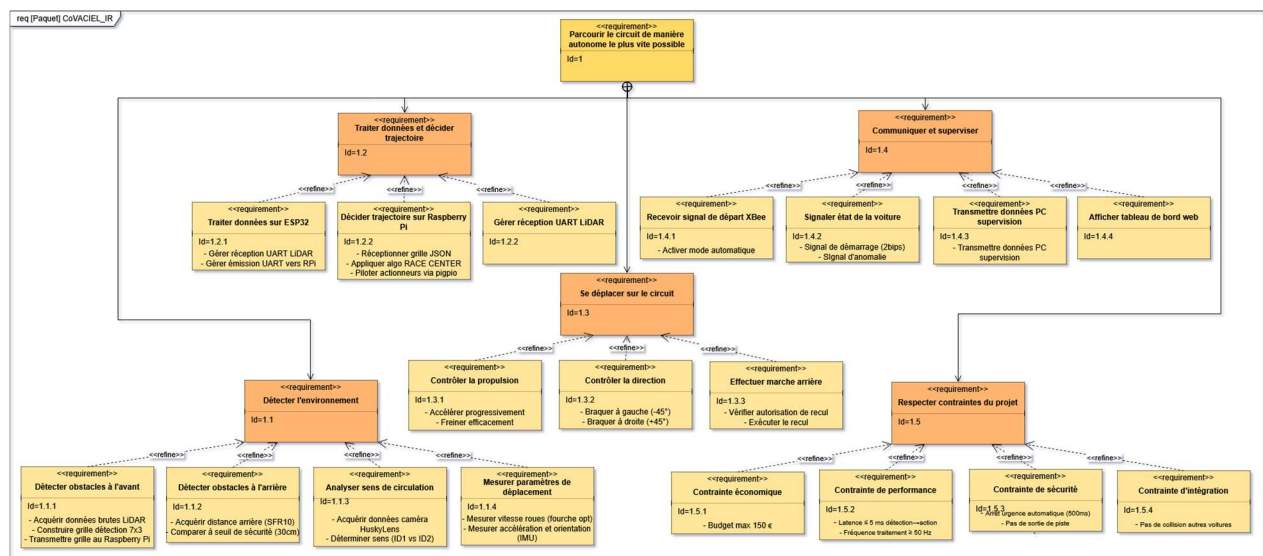


Figure 7 - Diagramme d'exigences SysML — Hiérarchie des exigences CoVACIEL

4. Parties communes

4.1 Répartition des tâches

Étudiant	Sous-systèmes BDD	Tâches détaillées
Jonathan JAEGER	Traitement (ESP32, Raspberry Pi)	Décodage RPLIDAR, grille 7×3, RACE_CENTER, commande PWM servo et ESC, surveillance, interface monitoring, optimisations
Quentin STUTZMANN	Carte HAT, Alimentation	Câblage complet capteurs et actionneurs, intégration mécanique châssis, tests de continuité
Hugo KRAWIEC	Capteurs (SRF10, XBee, Buzzer)	Ultrason arrière I2C, signal XBee, buzzer, sécurité marche arrière
Betul ARTI	Interface HMI Web	Flask, SQLite, WebSocket, Chart.js, sécurisation réseau

4.2 Cartes électroniques

Les deux cartes PCB ont été entièrement conçues et réalisées par notre professeur Emmanuel KEITH. L'équipe les a intégrées et câblées sur le véhicule.

Carte HAT (Interface centrale)	Carte Alimentation
- Connecteur Raspberry Pi 40 broches - Support ESP32 WROOM DevKit - Module XBee Pro - PCA9517D : adaptateur I2C 3,3V / 5V - Connecteurs LiDAR, I2C, fourche, servo, ESC - Cavaliers MCU/RPI : sélection de l'affectation des signaux - LEDs de diagnostic état système	- Régulateur 5V pour les microcontrôleurs - Connecteur batterie V_BAT (NiMH 7,2V) - Interrupteur général MSTBV2 - Condensateurs de filtrage (100 nF, 100 µF, 330 µF) - Fusible de protection - Boutons BP1, BP2 et RESET - Buzzer intégré et mesure de tension batterie (ADC)

4.3 Choix du microcontrôleur

Étape	Microcontrôleur	Versions	Statut
1	STM32F411	v1.0 à v1.5	Abandonné : RPLIDAR non activable malgré plusieurs semaines de débogage
2	ESP32-S3 DevKit	v1.6 à v2.7.1	Migration réussie, LiDAR stable, LED NeoPixel RGB
3	ESP32 WROOM	v2.7.2 et suivantes	Version définitive, imposée par la conception de la carte HAT

5. Partie Avant — Quentin STUTZMANN

Ma partie couvre la partie avant de la voiture CoVACIEL. Elle comprend l'intégration du LiDAR A2M12 sur l'ESP32 ainsi que l'intégration de la caméra HuskyLens sur le Raspberry Pi.

5.1 Intégration LiDAR

Caractéristiques et principe de fonctionnement

Le RPLIDAR A2M12 de Slamtec est le capteur principal du système de perception. Il utilise la triangulation laser pour mesurer les distances : un faisceau laser est émis et réfléchi par les obstacles. L'angle de réflexion est mesuré sur une matrice CMOS interne, ce qui permet de calculer la distance avec précision. Le capteur effectue une rotation complète à 10 Hz, produisant environ 4000 points de mesure par seconde sur 360°.

- Plage de mesure : 0,15 m à 12 m, précision $\pm 1\%$
- Rotation : 10 Hz (600 tours par minute), environ 4000 points par seconde
- Interface : liaison série UART à 115200 bauds, protocole Legacy Scan Slamtec
- Seuls les 180° avant sont exploités pour la détection des obstacles

Prise en main avec le logiciel Frame Grabber

Pour débiter l'intégration du LiDAR, j'ai utilisé le logiciel officiel Frame Grabber fourni par Slamtec. Cet outil permet de visualiser en temps réel les données brutes du RPLIDAR A2M12 sous forme de nuage de points polaire, sans avoir à développer de code. C'est la première étape indispensable pour valider que le capteur fonctionne correctement avant de l'intégrer à l'ESP32.

Le logiciel se connecte directement au LiDAR via un port COM (ici COM13 à 460 800 bauds). Il affiche un radar polaire avec des cercles concentriques représentant les distances (par pas de 1 000 mm), les points cyan correspondant aux obstacles détectés, et les lignes rouges marquant les zones d'alerte. L'en-tête indique en temps réel la fréquence de rotation (ici 12,5 Hz / 750 RPM) et la distance courante en millimètres.

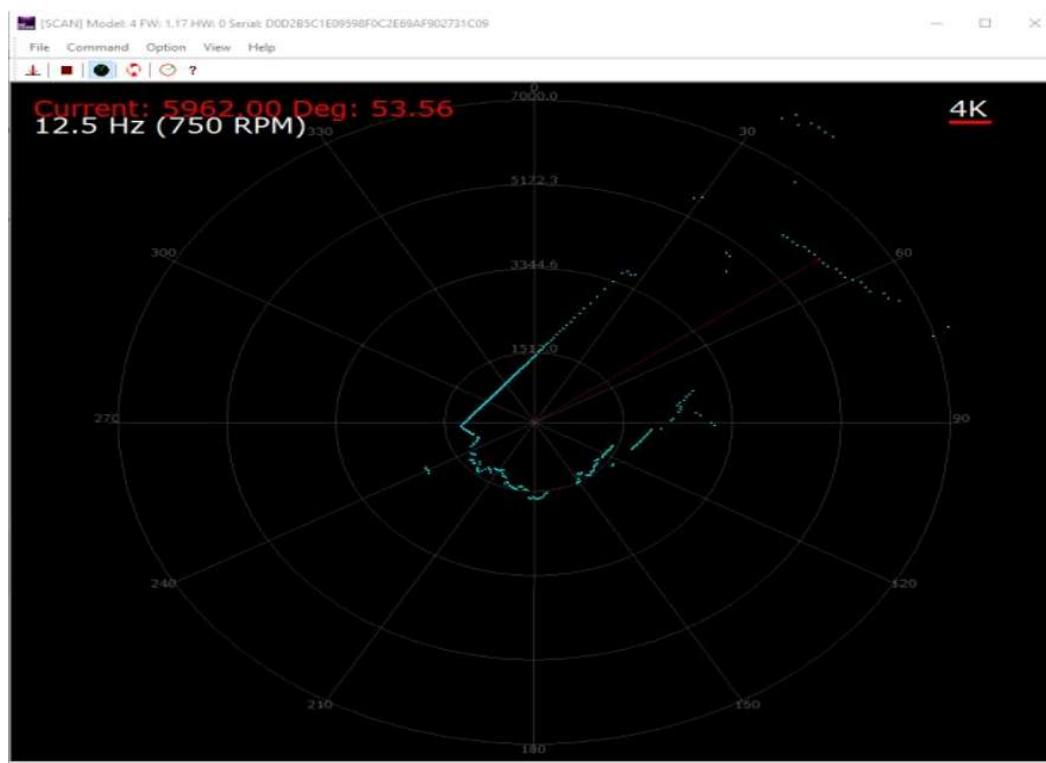


Figure 8 - Capture d'écran du logiciel Frame Grabber

L'interface Frame Grabber confirme le bon fonctionnement du LiDAR. On observe sur la capture que les obstacles de la salle (murs, mobilier) sont correctement représentés sur le radar. Les points cyan forment des arcs de cercle cohérents avec la géométrie réelle de l'environnement. Cette validation visuelle est essentielle avant de passer au décodage logiciel.

Programmes de test Python — LidarTest910 et LidarTest913

Avant d'intégrer le LiDAR sur l'ESP32, j'ai développé deux programmes de test en Python sur PC afin de valider le protocole de décodage et de visualiser les données en temps réel. Ces scripts utilisent la bibliothèque pyserial pour lire le flux UART et pygame pour l'affichage graphique.

LidarTest910_OK.py — Mode Mobilité (version stabilisée)

Ce programme est la version stabilisée après correction des problèmes rencontrés. Il ouvre le port série (COM13, 460 800 bauds) et envoie la commande de démarrage 0xA5 0x20 (commande SCAN du protocole Slamtec). Un thread dédié lit le flux série en continu et décode chaque paquet de 5 octets pour en extraire l'angle et la distance. L'affichage pygame montre un radar en mode mobilité : les points datant de plus de 0,5 seconde sont automatiquement effacés, ce qui évite les traînées lors des déplacements du capteur.

Extrait — LidarTest910_OK.py : Visualisation LiDAR en mode mobilité (Python / pygame)

```
# LidarTest910_OK.py - Décodage et affichage en mode mobilité (extrait clé)
# Lecture du buffer série en bloc (ser.in_waiting) pour éviter la saturation.
# Timeout série réduit à 0,001 s pour ne pas bloquer la boucle d'affichage.

def update_data(self):
    self.ser.write(bytearray([0xA5, 0x20])) # Commande SCAN (protocole Slamtec)
    while self.running:
        if self.ser.in_waiting > 100: # Lecture par bloc (>100 octets)
            raw = self.ser.read(self.ser.in_waiting)
            for i in range(len(raw) - 5):
                # Alignement sur l'octet de start (bit de synchro)
                if (raw[i+1] & 0x01) and not (raw[i] & 0x01):
                    angle_raw = (raw[i+1] >> 1) | (raw[i+2] << 7)
                    angle = angle_raw / 64.0
                    dist = (raw[i+3] | (raw[i+4] << 8)) / 4.0
                    if dist > 10:
                        angle_corr = (360 - angle) % 360 # Correction miroir
                        # Stockage avec horodatage pour supprimer les vieux
points
                        self.points[int(angle_corr)] = (dist, time.time())

# Dans la boucle d'affichage :
# Les points datant de plus de 0,5 s sont effacés -> pas de traînées en mobilité
if now - timestamp > 0.5:
    del lidar.points[angle]
    continue
```

LidarTest913.py — Mode Radar Pro (version améliorée)

Cette version intègre des filtres de qualité plus élaborés. Un filtre médian (sur un historique de 5 mesures par angle) élimine les points aberrants. Un filtre de densité radiale compare chaque point avec ses voisins angulaires ($\pm 1^\circ$) et rejette les points isolés (poussière, bruit). L'affichage utilise un fond bleu nuit, des points cyan et le double buffering pygame pour un rendu fluide à 40 FPS.

Extrait — LidarTest913.py : Visualisation LiDAR en mode Radar Pro avec filtres (Python / pygame)

```
# LidarTest913.py - Filtre médian et filtre de densité radiale (extraits clés)

# FILTRE MÉDIAN : historique de 5 mesures par angle
# Élimine les points aberrants (poussière, réflexions parasites)
self.history = {a: deque(maxlen=5) for a in range(360)}

if len(self.history[angle]) >= 3:
    med_dist = np.median(self.history[angle]) # Valeur centrale du buffer
    self.raw_points[angle] = (med_dist, time.time())

# FILTRE DE DENSITÉ RADIALE : comparaison avec les voisins angulaires ( $\pm 1^\circ$ )
# Un point isolé sans voisin proche est probablement du bruit -> rejeté
def get_filtered_scan(self):
    for angle in angles:
        dist, ts = self.raw_points[angle]
        neighbors = []
        for diff in [-1, 1]:
            n_angle = (angle + diff) % 360
            if n_angle in self.raw_points:
                n_dist, n_ts = self.raw_points[n_angle]
                if now - n_ts < 0.3:
                    neighbors.append(n_dist)
        # Rejet si le point s'écarte de plus de 20 cm de ses voisins
        if len(neighbors) >= 1:
```

```
if abs(dist - sum(neighbors)/len(neighbors)) < 200:
    final_points.append((angle, dist))
```

Difficultés rencontrées — Buffer série saturé

La principale difficulté rencontrée lors des tests Python a été la saturation du buffer série. Le RPLIDAR A2M12 émet environ 4 000 points par seconde, soit un débit de données très élevé. Si le programme ne lit pas le buffer assez rapidement, celui-ci se remplit entièrement, provoquant des désalignements de trame et des erreurs de décodage : des distances aberrantes ou des angles complètement faux apparaissaient sur le radar.

La solution adoptée a été de modifier le timeout du port série (réduit à 0,001 s dans LidarTest910 pour ne pas bloquer la boucle) et de lire ser.in_waiting octets d'un seul coup plutôt qu'octet par octet. Dans LidarTest910, la lecture ne s'effectue que lorsque plus de 100 octets sont disponibles. Dans LidarTest913, ce seuil est monté à 200 octets pour éviter les appels trop fréquents. Ces ajustements ont permis d'atteindre un affichage stable et fluide sans erreur de décodage.

Ces deux programmes de test ont ainsi servi de base de validation pour le décodage du protocole RPLIDAR Legacy Scan, avant de transposer le code sur l'ESP32 en C++.

Structure de la grille 7×3

Les données brutes du LiDAR (angles et distances) sont transformées en une grille de détection à 7 colonnes et 3 lignes :

Dimension	Description	Valeur
7 zones angulaires	L3, L2, L1, Centre, R1, R2, R3	De -90° à +90°, zones de 20 à 30° chacune
3 niveaux de distance	Seuils de danger croissants	URGENCE < 200 mm, RALENTI < 500 mm, ESQUIVE < 1200 mm
Fréquence de mise à jour	Recalcul à chaque scan partiel	50 Hz (toutes les 20 ms)
Format de transmission	Message JSON vers le Raspberry Pi	{"v":"...", "g":[[z0],[z1]...[z6]]}
Zones de décision	Zones qui déclenchent des actions	L1, Centre et R1 uniquement (L2, L3, R2, R3 sont informatifs)

Bug critique résolu — Décodage des angles

Un bug grave a été présent de la version 2.0 à la version 2.6 : le LiDAR ne détectait des obstacles qu'à l'arrière du véhicule. La cause était un décodage incorrect du protocole RPLIDAR Legacy Scan.

Formule incorrecte : `angle_q6 = (buf[2] << 8) | buf[1]`

Formule correcte : `angle_q6 = (buf[1] >> 1) | (buf[2] << 7)`

Dans le protocole Legacy Scan de Slamtec, le premier octet de l'angle (buf[1]) contient un bit de contrôle de validité en position 0 (le bit le moins significatif). Ce bit ne fait pas partie de la valeur de l'angle. En ne le retirant pas avant le calcul, la valeur obtenue était décalée et tous les angles pointaient vers l'arrière. La correction consiste à décaler cet octet d'un bit vers la droite avant de le combiner avec le second octet. Le problème a été identifié par comparaison directe avec la version 2.4, qui fonctionnait correctement.

Extraits de code clés

Les extraits suivants présentent les fonctions essentielles à l'acquisition LiDAR.

Extrait 1 — ESP32 : Décodage du protocole RPLIDAR Legacy Scan (correction du bit de contrôle)

```
// ESP32 — Décodage protocole RPLIDAR Legacy Scan
// Structure d'un paquet (5 octets) :
//   buf[0] : octet de qualité et bit de synchronisation
//   buf[1] : octet bas de l'angle — le bit 0 est un bit de CONTRÔLE, pas un bit d'angle !
//   buf[2] : octet haut de l'angle
//   buf[3] : octet bas de la distance
//   buf[4] : octet haut de la distance
void processLidarPacket(uint8_t* buf) {
    if ((buf[0] & 0x01) != ((buf[0] >> 1) & 0x01)) return; // synchro invalide
    // DÉCALAGE DROIT obligatoire pour retirer le bit de contrôle de buf[1]
    uint16_t angle_q6 = (buf[1] >> 1) | (buf[2] << 7); // 15 bits valides
    float angle = angle_q6 / 64.0f; // en degrés
    uint16_t dist_q2 = (buf[4] << 8) | buf[3];
    float dist_mm = dist_q2 / 4.0f;
    if (dist_mm > 0 && dist_mm < 12000)
        scan[(int)angle % 360] = (uint16_t)dist_mm; // uint16_t : -720 octets
    RAM vs float
}
```

Extrait 2 — ESP32 : Construction de la grille de détection 7×3

```
// ESP32 — Construction de la grille 7×3
// Pour chaque zone angulaire, on retient la distance minimale mesurée.
#define URGENGE_MM 200
#define RALENTI_MM 500
#define ESQUIVE_MM 1200
const int ZONES[8] = {-90, -60, -40, -20, 20, 40, 60, 90}; // L3..R3
void buildGrid(uint8_t grid[7][3]) {
    memset(grid, 0, 21);
    for (int z = 0; z < 7; z++) {
        uint16_t dMin = 65535;
        for (int a = ZONES[z]; a < ZONES[z+1]; a++) {
            int idx = ((a % 360) + 360) % 360;
            if (scan[idx] > 0 && scan[idx] < dMin) dMin = scan[idx];
        }
        grid[z][0] = (dMin < URGENGE_MM) ? 1 : 0;
        grid[z][1] = (dMin < RALENTI_MM) ? 1 : 0;
        grid[z][2] = (dMin < ESQUIVE_MM) ? 1 : 0;
    }
}
```

Extrait 3 — ESP32 vers Raspberry Pi : Envoi du JSON en une seule opération via la liaison série

```
// ESP32 vers Raspberry Pi — Envoi JSON atomique via UART
// snprintf() construit le message en mémoire, Serial.write() l'envoie en une
opération.
char buf[256];
void sendGrid(uint8_t g[7][3], const char* ver) {
    int n = snprintf(buf, sizeof(buf),
        "{\"v\":\"%s\",\"g\":[[%d,%d,%d],[%d,%d,%d],[%d,%d,%d],[%d,%d,%d],",
        "[%d,%d,%d],[%d,%d,%d],[%d,%d,%d]]",
        ver,
        g[0][0],g[0][1],g[0][2], g[1][0],g[1][1],g[1][2],
        g[2][0],g[2][1],g[2][2],
        g[3][0],g[3][1],g[3][2], g[4][0],g[4][1],g[4][2],
        g[5][0],g[5][1],g[5][2],
        g[6][0],g[6][1],g[6][2]);
    Serial.write((uint8_t*)buf, n); // écriture atomique
}
```

Difficultés rencontrées et solutions adoptées

Difficultés rencontrées	Solutions adoptées
- Identification de courts-circuits lors des premières mises sous tension - Espace très limité sur le châssis pour le passage des câbles - Longueurs de câbles initialement mal dimensionnées - Vérification de la continuité sur un grand nombre de connexions	- Validation incrémentale : chaque circuit testé isolément avant assemblage - Plan de montage préalable pour définir l'emplacement de chaque composant - Câbles de longueur maîtrisée, fixés par colliers rilsan - Tests systématiques au multimètre après chaque connexion

5.2 Caméra HuskyLens

Caractéristiques et principe de fonctionnement

La caméra IA HuskyLens de DFRobot a été intégrée pour assurer la reconnaissance visuelle des murs de la piste. Elle intègre un processeur dédié à la vision par ordinateur, soulageant ainsi le Raspberry Pi des lourds calculs de traitement d'image. Le capteur analyse une image de 320×240 pixels en temps réel.

Pour s'adapter aux variations d'éclairage et aux reflets potentiels sur la piste, la caméra a été configurée en mode Color Recognition avec un apprentissage multiple :

IDs 1 à 5 : Apprentissage de différentes nuances du mur GAUCHE (Rouge).

IDs 6 à 10 : Apprentissage de différentes nuances du mur DROIT (Vert).

Résolution d'analyse : 320 x 240 pixels (le centre est à x = 160).

Interface de communication : I2C direct via les broches SDA1/SCL1 du Raspberry Pi (Adresse 0x32).

Fréquence de rafraîchissement : Environ 20 Hz (boucle de lecture de 50 ms).

Stratégie de traitement et protocole I2C direct

Plutôt que d'utiliser une librairie constructeur parfois lourde ou instable, une communication I2C bas niveau a été développée en Python (via `smbus2`). Le Raspberry Pi envoie des requêtes brutes et décode la trame binaire renvoyée par la caméra (En-tête 0x55 0xAA 0x11, commande, données de la bounding box, et checksum).

La caméra renvoie pour chaque couleur reconnue un bloc contenant ses coordonnées x, y et ses dimensions w, h. La logique de détermination du sens de la marche repose sur deux étapes clés :

1. Filtrage : Isoler le plus grand bloc rouge et le plus grand bloc vert (pour ignorer le bruit de fond).

2. Comparaison spatiale : Si la coordonnée X du bloc rouge est inférieure à celle du bloc vert ($X_{\text{rouge}} < X_{\text{vert}}$), le rouge est à gauche de l'image : la voiture est dans le bon sens.

Extraits de code clés

Les extraits suivants présentent l'implémentation logicielle sur le Raspberry Pi pour extraire et analyser les données de la HuskyLens.

Extrait 4 — Raspberry Pi : Décodage bas niveau du protocole HuskyLens via I2C

```
# Raspberry Pi - Décodage de la réponse I2C de l'HuskyLens
# Permet de se passer d'une librairie externe complexe.
# Trame attendue : 0x55 0xAA 0x11 [Longueur] [Commande] [Données...] [Checksum]

def hl_get_blocks(bus):
    hl_send(bus, 0x20)    # Ping/Request
    time.sleep(0.02)
    hl_read(bus)

    hl_send(bus, 0x21)    # Demande des blocs
    time.sleep(0.02)

    blocks = []
    for _ in range(16):    # Lecture jusqu'à 10 IDs + marge de sécurité
        cmd, data = hl_read(bus)
        if cmd is None:
            break
        # Commande 0x2A = Block Data. La trame doit faire au moins 10 octets.
        if cmd == 0x2A and len(data) >= 10:
            x = data[0] | (data[1] << 8)
            y = data[2] | (data[3] << 8)
            w = data[4] | (data[5] << 8)
            h = data[6] | (data[7] << 8)
            bid = data[8] | (data[9] << 8)
            blocks.append({'x': x, 'y': y, 'w': w, 'h': h, 'id': bid})
    return blocks
```

Extrait 5 — Raspberry Pi : Fusion et filtrage des blocs par couleur

```
# Raspberry Pi - Sélection des blocs les plus pertinents
# La caméra peut détecter plusieurs blocs de la même couleur (reflets,
coupures).
# On ne conserve que le bloc ayant la plus grande surface (w * h).

RED_IDS = set(range(1, 6))    # ID 1 à 5 = MUR GAUCHE (rouge)
GREEN_IDS = set(range(6, 11)) # ID 6 à 10 = MUR DROIT (vert)

def group_blocks(blocks):
    best = {'rouge': None, 'vert': None}
    for b in blocks:
        bid = b['id']
        if bid in RED_IDS:
            key = 'rouge'
        elif bid in GREEN_IDS:
            key = 'vert'
        else:
            continue # ID inconnu (bruit), ignoré
        # Remplacement si c'est le premier bloc ou s'il est plus grand
        if best[key] is None or (b['w'] * b['h'] > best[key]['w'] *
best[key]['h']):
            best[key] = b
    return best
```

Extrait 6 — Raspberry Pi : Algorithme de détection du sens de la voiture

```
# Raspberry Pi - Logique d'analyse spatiale
# Le repère X part de 0 (gauche absolue) à 320 (droite absolue).

b_left = best['rouge']
b_right = best['vert']

if b_left and b_right:
    if b_left['x'] < b_right['x']:
        # Le bloc rouge a un X inférieur au vert : il est bien à gauche.
        sens_correct = True
        # Calcul du centrage du véhicule sur la piste
        mid = (b_left['x'] + b_right['x']) // 2
        offset = mid - (SCREEN_W // 2) # Écart par rapport au centre (160)
    else:
        # Le rouge est à droite du vert. La caméra regarde à l'envers.
        sens_correct = False
        alerte_mauvais_sens()
```

Difficultés rencontrées et solutions adoptées

Difficulté	Solution adoptée
Faux positifs ou détection de reflets générant de petits blocs parasites de couleur sur la piste.	Apprentissage de plusieurs ID pour une même couleur (5 IDs max par mur) et implémentation logicielle filtrant uniquement le bloc avec la plus grande surface ($w \times h$).
Instabilité de la communication I2C bloquant le programme principal si la caméra se déconnecte.	Création d'une tâche asynchrone dédiée à la lecture avec un mécanisme de verrou (<code>threading.Lock</code>) pour mettre à jour les données sans figer l'interface utilisateur ou le reste du contrôle.
Lourdeur et bugs des librairies tierces, souvent incompatibles avec les dernières versions Python du Raspberry Pi.	Réécriture complète de la communication en I2C brut (fonctions <code>hl_read</code> , <code>hl_send</code>) incluant un contrôle de redondance <code>hl_checksum(data)</code> garantissant l'intégrité des trames.
Validation difficile en conditions réelles sans écran branché sur le Raspberry Pi embarqué.	Développement d'une interface console en bibliothèque <code>curses</code> (v2.0) affichant une jauge ASCII graduée de 0 à 320 pour visualiser les détections de la caméra via SSH en temps réel.

6. Partie arrière — Hugo KRAWIEC

6.1 Réalisations

6.1.1 Capteur ultrason arrière — SRF10

Le capteur SRF10 assure la détection d'obstacles à l'arrière du véhicule autonome. Son choix résulte d'une analyse comparative approfondie avec le HC-SR04, dont les résultats sont présentés dans le tableau ci-dessous.

Caractéristiques techniques

Paramètre	Valeur
Interface	I2C — adresse 0x70
Alimentation / niveau logique capteur	5 V (adaptation via PCA9517D)
Plage de mesure	3 à 600 cm
Résolution	1 cm
Durée d'un cycle de mesure	≈ 65 ms
Ouverture angulaire	55° (3,7× supérieure au HC-SR04 à 15°)
Seuil de sécurité	Marche arrière interdite si distance < 30 cm

Tableau comparatif SRF10 vs HC-SR04

Critère	SRF10 ✓ Retenu	HC-SR04 ✗ Écarté
Interface	I2C (2 fils partagés)	Trigger / Echo (2 GPIO dédiés)
Angle de détection	55° — couverture large	15° — couverture étroite
Couverture angulaire relative	3,7× supérieure	Référence (1×)
Plage de mesure	3 à 600 cm	2 à 400 cm
Résolution	1 cm	≈ 0,3 cm (théorique)
Durée de mesure	≈ 65 ms	≈ 30 ms
Niveau logique	5 V (adaptation PCA9517D)	5 V (compatible direct)
Connexion au bus partagé	Oui — adressable (0x70)	Non — câblage individuel
Détection obstacles latéraux	Bonne (55°)	Insuffisante (15°)
Complexité d'intégration	Faible (I2C)	Moyenne (gestion timing précis)
Coût relatif	Plus élevé	Économique
Bilan	Retenu pour couverture et I2C	Écarté — zones d'ombre trop grandes

Le HC-SR04 présente un angle de détection de seulement 15°, insuffisant pour couvrir la largeur réelle du véhicule et les obstacles latéraux arrière. Le SRF10, avec ses 55°, garantit une couverture complète de la zone arrière.

Intégration sur l'ESP32

L'adaptateur de niveau logique PCA9517D est indispensable : l'ESP32 fonctionne en logique 3,3 V alors que le SRF10 requiert 5 V sur ses lignes I2C.

- Broche SDA : IO21 (ESP32) → SDA (SRF10 via PCA9517D)
- Broche SCL : IO22 (ESP32) → SCL (SRF10 via PCA9517D)
- Validation par scan I2C : détection confirmée à l'adresse 0x70

Logique de sécurité intégrée

Un seuil de 30 cm est défini dans le firmware : lorsque la distance mesurée est inférieure à cette valeur, la commande de marche arrière est immédiatement inhibée, quelle que soit la décision de l'algorithme de pilotage.

Code de pilotage du SRF10

Le séquençement de mesure respecte le protocole I2C du SRF10 : envoi de la commande de déclenchement (registre 0x00, valeur 0x51 pour mesure en centimètres), attente de la conversion (≈ 65 ms), puis lecture du registre de résultat sur 2 octets.

Extrait 1 — ESP32 : Code de pilotage du capteur SRF10 via I2C

```
#include <Wire.h>
#define SRF10_ADDR 0x70

void setup() {
    Serial.begin(115200);
    Wire.begin(21, 22);          // SDA=IO21, SCL=IO22
}

void loop() {
    // 1. Déclencher la mesure en cm
    Wire.beginTransmission(SRF10_ADDR);
    Wire.write(0x00);           // Registre de commande
    Wire.write(0x51);           // Mesure en centimètres
    Wire.endTransmission();

    delay(70);                  // Attente conversion (~65 ms + marge)

    // 2. Pointer le registre de résultat
    Wire.beginTransmission(SRF10_ADDR);
    Wire.write(0x02);           // Registre résultat (octet fort)
    Wire.endTransmission();

    // 3. Lire les 2 octets de distance
    Wire.requestFrom(SRF10_ADDR, 2);
    if (Wire.available() >= 2) {
        int distance = (Wire.read() << 8) | Wire.read();
        Serial.print("Distance : ");
        Serial.print(distance);
        Serial.println(" cm");
        // Sécurité : bloquer marche arrière si trop proche
        if (distance < 30) { /* stopper commande arrière */ }
    } else {
        Serial.println("Erreur lecture I2C");
    }
    delay(300);
}
```

6.1.2 Module XBee Pro — Signal de départ

Le module XBee Pro constitue le lien de communication sans fil entre la centrale d'organisation de la course et le véhicule. Il reçoit le signal de départ et le transmet à l'ESP32 via liaison série.

Configuration radio

Paramètre	Valeur
Protocole	IEEE 802.15.4 (firmware fourni par les organisateurs)
Fréquence	2,4 GHz
Débit série	9 600 bauds
Portée estimée en extérieur	≈ 100 m (suffisant pour le circuit)
Outil de configuration	XCTU (Digi International)
Connexion ESP32 — TX XBee	Broche IO25
Connexion ESP32 — RX XBee	Broche IO26

Trames de contrôle

Deux trames ASCII ont été définies en accord avec les organisateurs de la course :

Trame de départ : "\$GO" — déclenche le mode automatique du véhicule et active deux bips buzzer de confirmation.

Trame d'arrêt : "STOP" — interrompt immédiatement le mode automatique, indépendamment de l'état du véhicule.

La réception de la trame "\$GO" est traitée par une interruption logicielle sur l'UART2 de l'ESP32 (broches IO25/IO26). Dès détection, le firmware :

- Active le drapeau de mode automatique dans la tâche de navigation principale.
- Déclenche deux bips courts sur le buzzer pour confirmer la bonne réception.
- Enregistre l'horodatage de départ pour le calcul des performances.

6.1.3 Buzzer — Signalisation sonore

Le buzzer assure un retour sonore immédiat lors des événements critiques du système. Il est alimenté en 3,3 V via la carte HAT.

Événement	Signal sonore
Réception du signal de départ XBee ("GO")	2 bips courts (≈ 100 ms chacun)
Détection d'un défaut système	1 bip long (≈ 1 s)
Démarrage de l'ESP32 (POST)	1 bip court (auto-test)

6.2 Difficultés rencontrées et solutions adoptées

6.2.1 Choix du capteur arrière

L'analyse initiale reposait sur le HC-SR04, disponible en stock et bien documenté. Cependant, les tests en conditions réelles ont révélé que son faisceau étroit (15°) laissait des zones d'ombre importantes sur les côtés arrière du véhicule.

Le SRF10 a été retenu pour son angle de 55° et son interface I2C, plus robuste que le protocole trigger/echo du HC-SR04 en environnement bruyant électromagnétiquement.

6.2.2 Durée de mesure et contrainte temps-réel

La durée d'un cycle de mesure du SRF10 est de ≈ 65 ms. Si l'on envisageait 3 capteurs en cascade sur le bus I2C, le temps total atteindrait 195 ms — ce qui correspond à moins de 5 mesures par seconde, incompatible avec une réactivité suffisante.

La solution retenue pour la compétition est d'utiliser un seul capteur central, avec des mesures asynchrones via une tâche dédiée. Cette architecture permet de ne pas bloquer la boucle de pilotage principale tout en maintenant une fréquence de mesure suffisante.

6.2.3 Migration STM32 → ESP32

En cours de développement, le choix du microcontrôleur principal a évolué d'un STM32 vers un ESP32 (décision commune avec Jonathan JAEGER). Cette migration a impliqué les adaptations suivantes :

- Remappage des broches I2C : PB8/PB9 (STM32) → IO21/IO22 (ESP32).
- Ajout du PCA9517D pour la gestion des niveaux logiques 5 V/3,3 V.
- Validation de la communication par scan I2C : adresse 0x70 détectée et opérationnelle.
- Réécriture du code de pilotage en utilisant la bibliothèque Wire.h de l'Arduino Framework pour ESP32.

6.2.4 Configuration du réseau XBee

La configuration du module XBee avec le logiciel XCTU a nécessité de s'assurer que le firmware imposé par les organisateurs (IEEE 802.15.4) était correctement flashé et que les paramètres réseau (PAN ID, canal) correspondaient exactement à ceux du module coordinateur utilisé par les juges.

7. Pilotage autonome — Jonathan JAEGER

Ma partie couvre la chaîne de pilotage côté Raspberry Pi : réception et traitement des données JSON transmises par l'ESP32, algorithme de décision RACE_CENTER, commande PWM des actionneurs, interface de monitoring, surveillance automatique et validation du système.

7.1 Présentation du système côté Raspberry Pi

Le Raspberry Pi 4 Model B est le centre décisionnel du véhicule. Il reçoit la grille de détection calculée par l'ESP32, applique l'algorithme RACE_CENTER et génère les commandes PWM pour les actionneurs.

Composant	Rôle	Interface
Raspberry Pi 4 Model B	Réception JSON, algorithme RACE_CENTER, commande servo et ESC	UART depuis ESP32, GPIO via pigpio
Liaison UART (/dev/ttyS0)	Réception de la grille 7×3 en JSON depuis l'ESP32 à 50 Hz	115200 bauds, GPIO 14/15 (broches 8/10)
Servomoteur Carson CS-9	Contrôle de la direction, $\pm 45^\circ$	PWM sur GPIO 12, alimenté par BEC de l'ESC
Variateur ESC TAMIYA	Contrôle de la propulsion, 0 à 100%	PWM sur GPIO 13
pigpio	Bibliothèque de génération PWM matérielle	Démon système, précision μ s
Interface curses	Monitoring temps réel dans le terminal SSH	Rafraîchissement à 50 Hz

Flux de traitement : ESP32 WROOM (grille 7×3 JSON) → UART → Raspberry Pi 4 (RACE_CENTER) → GPIO PWM → Servo + ESC

7.2 Réception et traitement des données JSON

L'ESP32 transmet la grille de détection 7×3 toutes les 20 ms via la liaison série UART. Sur le Raspberry Pi, la réception est assurée par une tâche parallèle dédiée, afin de ne pas bloquer la boucle de décision principale.

Paramètre	Valeur	Remarque
Liaison physique	UART /dev/ttyS0 à 115200 bauds	Activé dans /boot/firmware/config.txt
Format reçu	JSON compact	Exemple : {"v": "2.7.2", "g": [[0,0,0]...[0,0,0]]}
Fréquence de réception	50 Hz (toutes les 20 ms)	Latence totale détection → décision < 5 ms
Structure partagée	Dictionnaire Python protégé par un verrou	Accès concurrent sécurisé entre tâches
Gestion des erreurs	Compteur d'erreurs JSON affiché en temps réel	Trames invalides ignorées sans blocage

Extrait 4 — Raspberry Pi : Tâche de réception et décodage JSON

```
# Raspberry Pi – Tâche de réception UART et décodage JSON
# Un verrou (threading.Lock) protège l'accès aux données partagées.
import serial, json, threading
ser = serial.Serial('/dev/ttyS0', 115200, timeout=0.1)
shared = {}
verrou = threading.Lock()
def tache_reception():
    nb_erreurs = 0
    while True:
        if ser.in_waiting > 0:
            try:
                data = json.loads(ser.readline().decode('utf-8').strip())
                with verrou:
                    shared['grille'] = data['g']
                    shared['version'] = data.get('v', '?')
                    shared['erreurs'] = nb_erreurs
            except (json.JSONDecodeError, UnicodeDecodeError):
                nb_erreurs += 1
            except KeyError:
                pass
    t = threading.Thread(target=tache_reception, daemon=True)
    t.start()
```

Le dictionnaire partagé contient à tout instant la dernière grille valide reçue, la version du firmware ESP32 et le compteur d'erreurs. La tâche principale y accède via le verrou pour garantir la cohérence des données.

7.3 Algorithme de décision RACE_CENTER

Simulation préalable et sélection du profil

Avant de déployer un algorithme sur le véhicule réel, nous avons développé une simulation en Python (bibliothèque pygame) reproduisant le comportement du véhicule sur un circuit virtuel avec parois et obstacles. Cette approche permet de comparer des profils en conditions contrôlées, sans risque pour le matériel, et de sélectionner objectivement le plus performant.

Profil testé	Comportement observé sur 3 tours	Verdict
RACE_AGGRESSIF	Vitesse maximale, corrections minimales. Collisions fréquentes, impossible de terminer.	Rejeté
SAFE_AGGRESSIF	Corrections larges à grande vitesse. Instabilité importante dans les virages serrés.	Rejeté
SAFE_CENTERED	Comportement très sécurisé mais vitesse insuffisante pour être compétitif.	Rejeté
RACE_CENTER	Meilleur compromis vitesse et sécurité. Trois tours complétés sans collision.	✅ Retenu

Règles de décision

À chaque cycle de 20 ms, RACE_CENTER lit la grille 7×3 reçue et calcule l'angle de braquage et la vitesse cible selon une hiérarchie de priorités :

Situation	Condition de déclenchement	Angle	Vitesse
Urgence centre	Obstacle < 200 mm (Centre URGENCE)	±45° vers côté libre	20%
Urgence bilatérale	Centre, L1 et R1 tous en URGENCE	0°	0% (arrêt)
Ralenti centre	Obstacle 200 à 500 mm (Centre RALENTI)	±35° vers côté libre	40%
Ralenti bilatéral	Centre, L1 et R1 tous en RALENTI	0°	0% (arrêt)
Esquive lointaine	Obstacle 500 à 1200 mm (Centre ESQUIVE)	±15° vers côté libre	80%
Voie libre	Aucun obstacle dans L1, Centre, R1	0°	100%

Extrait 5 — Raspberry Pi : Algorithme de décision RACE_CENTER

```
# Raspberry Pi – Algorithme de décision RACE_CENTER
# Entrée : grille 7x3 | Sortie : (angle°, vitesse%, label, priorité)
def decider_action(grille):
    Llu,Llr = grille[2][0],grille[2][1] # L1
    Cu, Cr = grille[3][0],grille[3][1] # Centre
    Rlu,Rlr = grille[4][0],grille[4][1] # R1
    Ces = grille[3][2] # Centre ESQUIVE
    if Cu: # Urgence centre : obstacle < 200 mm
        if not(Llu or Llr): return -45,20,"URGENCE, GAUCHE MAX", "URGENCE"
        elif not(Rlu or Rlr): return +45,20,"URGENCE, DROITE MAX", "URGENCE"
        else: return 0, 0,"ENTOURÉ, ARRÊT", "URGENCE"
    if Cr: # Ralenti centre : obstacle 200-500 mm
        if not(Llu or Llr): return -35,40,"RALENTI, GAUCHE", "CRITIQUE"
        elif not(Rlu or Rlr): return +35,40,"RALENTI, DROITE", "CRITIQUE"
        else: return 0, 0,"BLOQUÉ, ARRÊT", "URGENCE"
    if Ces: # Esquive : obstacle 500-1200 mm
        if not grille[2][2]: return -15,80,"ESQUIVE, LÉGÈRE GAUCHE","FAIBLE"
        if not grille[4][2]: return +15,80,"ESQUIVE, LÉGÈRE DROITE","FAIBLE"
    return 0,100,"VOIE LIBRE","LIBRE"
```

Rampe d'accélération et de freinage

La vitesse évolue progressivement vers la valeur cible. Un cycle correspond à une itération de la boucle principale, soit environ 20 ms. L'objectif est d'éviter les à-coups mécaniques sur les actionneurs.

Situation	Taux appliqué	Durée pour 0% à 100%
Accélération vers vitesse faible (< 50%)	+ 2,0% par cycle	Environ 50 cycles soit 1 seconde
Accélération vers vitesse moyenne (< 80%)	+ 4,0% par cycle	Environ 25 cycles soit 500 ms
Accélération pleine vitesse ($\geq 80\%$)	+ 8,0% par cycle	Environ 13 cycles soit 260 ms
Freinage normal	- 5,0% par cycle	Environ 20 cycles soit 400 ms
Freinage fort (situation critique)	- 15,0% par cycle	Environ 7 cycles soit 140 ms
Freinage d'urgence (obstacle très proche)	- 50,0% par cycle	2 cycles soit moins de 40 ms

Marche arrière

- Activation : Centre, L1 et R1 tous en URGENCE pendant au moins 2 secondes consécutives
- Comportement : recul à -30% de la vitesse nominale, braquage à $\pm 15^\circ$ vers le côté le plus dégagé
- Sortie : Centre libre en RALENTI, ou dépassement du délai de 5 secondes
- Condition de sécurité : marche arrière autorisée uniquement si le SRF10 arrière mesure plus de 30 cm

7.4 Commande des actionneurs via pigpio

La bibliothèque pigpio génère les signaux PWM directement depuis le Raspberry Pi avec une précision de l'ordre de la microseconde. Elle fonctionne en mode démon système (sudo pigpiod) et est accessible depuis Python.

Actionneur	GPIO	Plage PWM	Signal neutre
Servomoteur Carson CS-9 (direction)	GPIO 12	1000 μ s (-45°) à 2000 μ s (+45°)	1500 μ s (tout droit)
Variateur ESC TAMIYA (propulsion)	GPIO 13	1500 μ s (0%) à 2000 μ s (100%)	1500 μ s (arrêt)

Extrait 6 — Raspberry Pi : Commande PWM servo et ESC via pigpio

```
# Raspberry Pi - Commande PWM servo et ESC via pigpio
# GPIO 12 = servo direction | GPIO 13 = variateur ESC propulsion
# IMPORTANT : le servo est alimenté par le BEC de l'ESC (pas par le Raspberry Pi).
# Seul le câble signal (PWM) est connecté au GPIO 12.
import pigpio
pi = pigpio.pi()
vitesse_act = 0 # état interne de la rampe
def regler_servo(angle):
    # -45° -> 1000 µs | 0° -> 1500 µs | +45° -> 2000 µs
    pi.set_servo_pulsewidth(12, int(1500 + (max(-45,min(45,angle))/45.0)*500))
def mettre_a_jour_vitesse(cible, priorite):
    global vitesse_act
    freinage = {"URGENCE":50.0,"CRITIQUE":15.0}.get(priorite,5.0)
    if cible > vitesse_act:
        taux = 8.0 if cible>=80 else 4.0 if cible>=50 else 2.0
        vitesse_act = min(vitesse_act+taux, cible)
    else:
        vitesse_act = max(vitesse_act-freinage, cible)
    pi.set_servo_pulsewidth(13, int(1500+vitesse_act/100.0*500))
```

7.5 Alimentation du servomoteur — problème et solution

Problème rencontré lors des tests sur piste

Lors des premières sessions de test, nous avons observé des saccades importantes et irrégulières sur le servomoteur de direction. L'algorithme calculait correctement les angles, mais les mouvements physiques du servo n'étaient pas conformes aux consignes.

Symptômes observés

- Mouvements brusques et non répétables du servomoteur, indépendants des consignes calculées
- Perturbations ponctuelles sur la liaison UART (augmentation du compteur d'erreurs JSON)
- Problème aggravé quand le moteur de propulsion était fortement sollicité simultanément
- Sur banc de test avec alimentation externe dédiée pour le servo, les saccades disparaissaient

Diagnostic

Le servo Carson CS-9 était alimenté par les broches 5V du Raspberry Pi. Lors des déplacements angulaires rapides, il peut appeler jusqu'à 800 mA en pointe. Cette consommation provoquait des chutes de tension sur le bus 5V interne du Raspberry Pi, perturbant la génération PWM et la liaison UART.

Solution : alimentation par le BEC de l'ESC

Nous avons connecté le servo directement sur le BEC (Battery Eliminator Circuit) du variateur ESC. Le BEC est un régulateur 5V intégré à l'ESC, conçu spécifiquement pour alimenter les servomoteurs, capable de fournir jusqu'à 2 A en continu.

Modifications réalisées

- Déconnexion du câble d'alimentation du servo (+5V et masse) depuis le Raspberry Pi
- Connexion du +5V et de la masse du servo vers le BEC de l'ESC
- Conservation du seul câble signal PWM entre GPIO 12 (Raspberry Pi) et l'entrée signal du servo
- Ajout d'une masse commune entre le Raspberry Pi et l'ESC pour la cohérence du signal PWM

Résultats obtenus

- Disparition complète des saccades dès les premiers tests après la modification
- Stabilité totale de la liaison UART : zéro erreur JSON sur 10 minutes de test continu
- Le Raspberry Pi n'est plus perturbé par les appels de courant du servo

Composant	Source d'alimentation	Tension
Raspberry Pi 4	Carte alimentation (régulateur 5V via carte HAT)	5V
ESP32 WROOM	Régulateur interne du Raspberry Pi (via carte HAT)	3,3V
RPLIDAR A2M12	Carte alimentation	5V
Servomoteur Carson CS-9	BEC du variateur ESC	5V (jusqu'à 2A)
Variateur ESC (moteur)	Batterie NiMH directement	7,2V
Capteurs I2C (SRF10, HuskyLens)	Carte alimentation via PCA9517D	5V / 3,3V

7.6 Interface de monitoring temps réel

Pour superviser le système pendant les tests et faciliter le diagnostic, j'ai développé une interface de monitoring en mode texte (bibliothèque curses de Python). Elle s'affiche dans le terminal SSH du Raspberry Pi et se rafraîchit à 50 Hz.

Élément affiché	Description
Grille 7×3	Représentation visuelle des 21 cellules (■ = obstacle, · = libre) par niveau
Angle de direction	Valeur en degrés et barre graphique proportionnelle
Vitesse actuelle	Valeur en pourcentage et barre de progression sur 40 caractères
Action en cours	Label de la décision RACE_CENTER
Niveau de priorité	Indicateur : URGENCE, CRITIQUE, FAIBLE, LIBRE
Compteur d'erreurs JSON	Nombre de trames incorrectes depuis le démarrage
Version ESP32	Numéro de version du firmware reçu dans chaque trame

Extrait 7 — Raspberry Pi : Interface de monitoring curses

```
# Raspberry Pi - Interface de monitoring temps réel (bibliothèque curses)
# Affichage dans le terminal SSH, rafraîchi à 50 Hz.
import curses
def afficher(stdscr, grille, angle, vitesse, label, priorite, erreurs):
    icones={"URGENCE":"⬤", "CRITIQUE":"◐", "FAIBLE":"◑", "LIBRE":"◒"}
    stdscr.clear()
    stdscr.addstr(0,0,"CoVACIEL - Monitoring",curses.A_BOLD)
    stdscr.addstr(1,0,f"Erreurs JSON : {erreurs} | {icones.get(priorite,'?')}")
    {priorite})
    niveaux=["URG","RAL","ESQ"]
    zones=["L3","L2","L1","CTR","R1","R2","R3"]
    for ni,niv in enumerate(niveaux):
```

```

    ligne=f"{niv} |"+"".join(" ■" if grille[z][ni] else " ." for z in
range(7))
    stdscr.addstr(3+ni,0,ligne)
    stdscr.addstr(6,0," "+"".join(f" {z:>2}" for z in zones))
    barre="█"*int(vitesse/2.5)+"."*(40-int(vitesse/2.5))
    stdscr.addstr(8, 0,f"Direction : {angle:+5.1f}°")
    stdscr.addstr(9, 0,f"Vitesse : {vitesse:5.1f}% [{barre}]")
    stdscr.addstr(10,0,f"Action : {label}")
    stdscr.refresh()

```

7.7 Surveillance automatique des communications

Si aucune trame JSON valide n'est reçue depuis l'ESP32 pendant plus de 500 ms, le moteur de propulsion est arrêté immédiatement et le servo remis en position centrale. Le système attend la reprise de la communication avant de reprendre le pilotage.

Extrait 8 — Raspberry Pi : Surveillance automatique de la communication

```

# Raspberry Pi – Surveillance automatique de la communication
# Si aucune trame valide n'est reçue pendant plus de 500 ms, le moteur s'arrête.
import time
TIMEOUT = 0.500
derniere_trame = time.time()
def boucle_principale():
    global derniere_trame
    while True:
        with verrou:
            grille = shared.get('grille', None)
            erreurs = shared.get('erreurs', 0)
            if grille is None:
                continue
            if (time.time() - derniere_trame) > TIMEOUT:
                mettre_a_jour_vitesse(0,"URGENCE") # arrêt immédiat
                regler_servo(0)
                continue
            derniere_trame = time.time()
            angle,vitesse,label,prio = decider_action(grille)
            regler_servo(angle)
            mettre_a_jour_vitesse(vitesse,prio)

```

7.8 Méthodologie de test et validation

Tests unitaires de l'algorithme (200 grilles aléatoires)

Avant le déploiement sur piste, j'ai développé un script de validation automatique générant 200 grilles 7×3 aléatoires et vérifiant que RACE_CENTER respecte les contraintes suivantes :

- La vitesse est toujours comprise entre 0% et 100%
- L'angle est toujours compris entre -45° et +45°
- Si le Centre est en URGENCE, la vitesse ne dépasse jamais 20%
- Si Centre, L1 et R1 sont tous en URGENCE, la vitesse est obligatoirement 0%
- La priorité retournée correspond au niveau le plus élevé présent dans la grille

Les 200 tests ont tous été validés sans aucune erreur.

Tests sur piste par étapes successives

Étape de test	Objectif	Résultat
Servo seul, véhicule soulevé	Vérifier la réponse PWM aux consignes d'angle	Angles corrects, sans oscillation
ESC seul, piste droite	Vérifier la rampe d'accélération et le freinage	Progression conforme au tableau des taux
Grille JSON simulée, statique	Vérifier les décisions RACE_CENTER en conditions réelles	Décisions correctes sur toutes les situations
Système complet, circuit simple	Premier passage autonome complet	Réussi, saccades servo identifiées puis corrigées (section 7.5)
Système complet, circuit officiel	Validation en conditions de compétition	Comportement conforme au profil RACE_CENTER

7.9 Optimisations réalisées côté Raspberry Pi

Optimisation	Description	Bénéfice
Arrêt automatique sur perte de communication	Si aucune trame JSON valide n'est reçue pendant plus de 500 ms, le moteur s'arrête	Sécurité : aucun comportement incontrôlé possible
Protection des accès concurrents	Un verrou unique protège le dictionnaire partagé entre la tâche de réception et la boucle principale	Cohérence des données garantie
Compteur d'erreurs JSON	Chaque trame invalide est comptée et affichée dans l'interface de monitoring	Diagnostic rapide des problèmes liaison
Affichage version firmware ESP32	La version reçue à chaque trame est affichée en temps réel	Vérification du bon firmware chargé
Interface curses complète	Toutes les informations utiles sur un seul écran, rafraîchi à 50 Hz	Supervision efficace pendant les tests
Validation sur 200 grilles aléatoires	Script de test automatique avant tout déploiement terrain	Certification de la robustesse algorithmique

7.10 Historique des versions

Version	Date	Description
v1.0	08 janv. 2026	Démarrage du projet. Architecture initiale. Premiers tests avec le STM32F411.
v1.5	23 janv. 2026	Abandon du STM32F411 : RPLIDAR non activable. Décision de migration vers l'ESP32.
v1.6	05 fév. 2026	Migration vers l'ESP32-S3 DevKit. Grille 3×3 fonctionnelle. Première détection LiDAR stable.
v2.0	19 fév. 2026	Passage à la grille 5×3. Révision complète du câblage. Première intégration sur le châssis.
v2.4	05 mars 2026	Codes d'erreur LED. Accélération progressive. Tests de rampe validés.
v2.7.0	19 mars 2026	Grille étendue à 7×3. Algorithme RACE_CENTER. Interface curses. Marche arrière.
v2.7.1	26 mars 2026	Correction du bug de décodage des angles (bit de contrôle). Détection correcte à 360°.
v2.7.1.1	09 avr. 2026	Migration vers l'ESP32 WROOM (carte HAT). Adaptation à l'API Arduino 3.x.
v2.7.2	23 avr. 2026	Optimisations RAM, surveillance des communications, protection des accès concurrents, alimentation servo corrigée. Version finale.

8. Dashboard Web — Betul ARTI

Ma partie couvre la réalisation complète du site web statistique dynamique du projet CoVACIEL, ainsi que la gestion du planning général. Ce site permet de visualiser les données de course, de superviser la voiture en temps réel, et de présenter le projet à l'extérieur.

8.1 Rôle et objectifs du site

Le site web répond à trois besoins distincts identifiés dès le début du projet :

- Infrastructure réseau : configurer un serveur web sécurisé directement sur la Raspberry Pi 400 de la voiture.
- Stockage : concevoir une base de données MySQL enregistrant l'historique de chaque course (vitesse, consommation, collisions, distance).
- Interface (IHM) : développer une page web dynamique transformant ces données brutes en statistiques visuelles via des graphiques interactifs Chart.js.

Le site est structuré en trois niveaux d'accès :

Niveau d'accès	Pages concernées	Contenu
Public	Accueil, Équipe, Lycée, Statistiques	Présentation du projet, historique des courses, graphiques
Authentifié	Login → Dashboard	Monitoring temps réel via WebSocket, données capteurs en direct
Administrateur	/admin/users	Gestion des comptes utilisateurs, droits d'accès

Détail des cinq pages du site :

Page	Accès / Contenu
Accueil (index.html)	Public — Présentation du projet CoVACIEL, objectifs, équipe
Équipe (equipe.html)	Public — Présentation des 4 membres et des rôles
Présentation Lycée	Public — Lycée Heinrich Nessel (page séparée suite demande professeur)
Statistiques (statistiques.html)	Public — 4 cartes globales + 3 graphiques Chart.js alimentés par MySQL
Login + Dashboard (dashboardprv.html)	Authentifié — Monitoring temps réel WebSocket, 3 colonnes

Performance (Fiche E6)	Attendu	Réalisé
Installation d'un serveur web avec SGBD MySQL ou MariaDB	OUI	✓ OUI
Création de la base de données pour stocker les statistiques de course	OUI	✓ OUI
Création d'une page web dynamique	OUI	✓ OUI

8.2 Technologies utilisées

L'application repose sur plusieurs technologies complémentaires, fonctionnant ensemble sans connexion Internet sur le réseau hotspot de la Raspberry Pi.

Technologie	Rôle dans le projet
Python + Flask (app.py)	Serveur web principal — routes HTTP, sessions, connexion MySQL, émission WebSocket
HTML5 & CSS3	Structure et design du site — thème aurora inspiré Formule 1 (fond violet, cyan)
JavaScript + Socket.IO	Gestion du temps réel côté client — mise à jour instantanée des données capteurs
MySQL / MariaDB	Base de données — 3 tables : utilisateurs, courses, télémétrie
phpMyAdmin	Interface graphique web d'administration de la base de données
UwAmp (PC)	Environnement de développement local avant déploiement sur Raspberry Pi
Flask-SocketIO	Extension Flask pour WebSocket bidirectionnel — push des données vers le dashboard



Figure 9 - Logo UwAmp — Environnement de développement local (Apache, MySQL, PHP)

8.3 Conception de la base de données

La première étape de réalisation a été de définir la structure de la base de données avant même d'écrire la moindre ligne de code HTML. La base CoVACIEL contient trois tables interdépendantes.

Modèle logique des données (MLD)

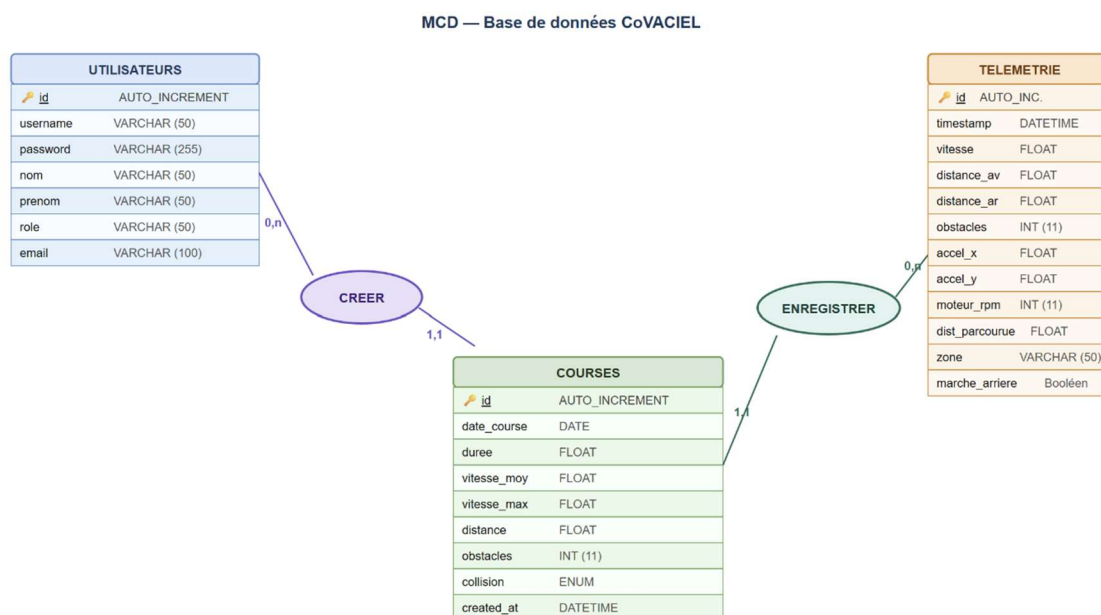


Figure 10 - MLD de la base de données CoVACIEL — Tables utilisateurs, courses et télémétrie

Table utilisateurs

Cette table stocke les comptes permettant l'accès authentifié au dashboard privé. Un compte administrateur par défaut est créé à l'installation. Les mots de passe sont stockés sous forme de hash SHA-256.

id	username	password	nom	prenom	role	email	telephone	linkedin
1	quentin	a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e99...	Stutzmann	Quentin	IR1 – Fonctions Front	quentin@email.com	+33600000000	https://linkedin.com/in/quentin-stutzmann
2	hugo	a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e99...	Krawiec	Hugo	IR2 – Fonctions Back	hugo@email.com	+33600000000	https://linkedin.com/in/hugokrawiec-7a76b8369
3	jonathan	a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e99...	Jaeger	Jonathan	IR3 – Traction & Pilotage	jonathan@email.com	+33600000000	https://linkedin.com/in/jonathajaeger-675729259
4	betul	a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e99...	Arti	Betul	IR4 – Site Web & Stats	betul@email.com	+33600000000	https://linkedin.com/in/betularti-01b036348

Figure 11 - Table utilisateurs dans phpMyAdmin — Structure et champs

Table courses

Cette table est le cœur du système statistique. Sa structure a été définie en collaboration avec tous les membres de l'équipe. Elle enregistre les métriques globales de chaque course : vitesse moyenne, consommation, taux de réussite, collisions évitées.

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Extra	Action
1	id	int(11)			Non	Aucune	AUTO_INCREMENT	Modifier Supprimer Primaire Unique
2	date_course	date			Non	Aucune		Modifier Supprimer Primaire Unique
3	duree	float			Oui	NULL		Modifier Supprimer Primaire Unique
4	vitesse_moy	float			Oui	NULL		Modifier Supprimer Primaire Unique
5	vitesse_max	float			Oui	NULL		Modifier Supprimer Primaire Unique
6	distance	float			Oui	NULL		Modifier Supprimer Primaire Unique
7	obstacles	int(11)			Oui	0		Modifier Supprimer Primaire Unique
8	collision	enum('oui', 'non')	utf8mb4_unicode_ci		Oui	non		Modifier Supprimer Primaire Unique
9	commentaire	text	utf8mb4_unicode_ci		Oui	NULL		Modifier Supprimer Primaire Unique
10	created_by	varchar(50)	utf8mb4_unicode_ci		Oui			Modifier Supprimer Primaire Unique
11	created_at	datetime			Oui	CURRENT_TIMESTAMP		Modifier Supprimer Primaire Unique

Figure 12 - Table courses dans phpMyAdmin — Données statistiques par course

Table télémétrie

La table télémétrie enregistre les mesures capteurs en temps réel envoyées par la voiture via WebSocket. Elle contient 13 champs : vitesse, distance avant, angle servo, zone active, horodatage et identifiant de course.

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Extra	Action
1	id	int(11)			Non	Aucune	AUTO_INCREMENT	Modifier Supprimer Primaire
2	timestamp	datetime			Oui	CURRENT_TIMESTAMP		Modifier Supprimer Primaire
3	vitesse	float			Oui	0		Modifier Supprimer Primaire
4	distance_av	float			Oui	0		Modifier Supprimer Primaire
5	distance_ar	float			Oui	0		Modifier Supprimer Primaire
6	obstacles	int(11)			Oui	0		Modifier Supprimer Primaire
7	accel_x	float			Oui	0		Modifier Supprimer Primaire
8	accel_y	float			Oui	0		Modifier Supprimer Primaire
9	gyro_z	float			Oui	0		Modifier Supprimer Primaire
10	moteur_rpm	int(11)			Oui	0		Modifier Supprimer Primaire
11	distance_parcourue	float			Oui	0		Modifier Supprimer Primaire
12	zone	varchar(50)	utf8mb4_unicode_ci		Oui	LIBRE		Modifier Supprimer Primaire
13	marche_arriere	tinyint(1)			Oui	0		Modifier Supprimer Primaire

Figure 13 - Table télémétrie dans phpMyAdmin — Mesures capteurs en temps réel

Migration PC → Raspberry Pi

Le processus de déploiement de la base de données du PC vers la Raspberry Pi se déroule en quatre étapes :

- Sur le PC : phpMyAdmin → onglet « Exporter » → format SQL → téléchargement du fichier covaciel.sql.
- Transfert : copie du fichier covaciel.sql sur clé USB Philips.
- Sur la Raspberry Pi : phpMyAdmin → onglet « Importer » → sélection du fichier → exécution.
- Vérification : les tables courses, télémétrie et utilisateurs apparaissent dans phpMyAdmin.

8.4 Diagrammes de conception

Les diagrammes UML modélisent le comportement et les acteurs du système.

Diagramme de séquence

Le diagramme de séquence décrit le flux d'interactions lors d'une session de monitoring. Il implique quatre participants : l'Utilisateur (élève/jury), le Navigateur (JavaScript), le Serveur Flask et la Voiture (ESP32/Raspberry Pi pilotage).

Phase 1 – Connexion réseau (Wi-Fi hotspot) : L'utilisateur détecte le réseau RaspberryCovaciellR-HN (WPA3) et s'y connecte. La Raspberry Pi lui attribue une IP via DHCP (10.42.0.x). Le site est accessible à l'adresse `http://10.42.0.1:5000`.

Phase 2 – Authentification (Flask/MySQL) : L'accès au dashboard est protégé. Le serveur effectue une requête `SELECT` dans la table utilisateurs, compare le hash SHA-256 du mot de passe saisi, et crée une session Flask si les identifiants sont corrects.

Phase 3 – Monitoring temps réel : La voiture envoie ses données via WebSocket (événement telemetry). Flask-SocketIO les reçoit, les insère en base, et les redistribue instantanément à tous les dashboards ouverts.

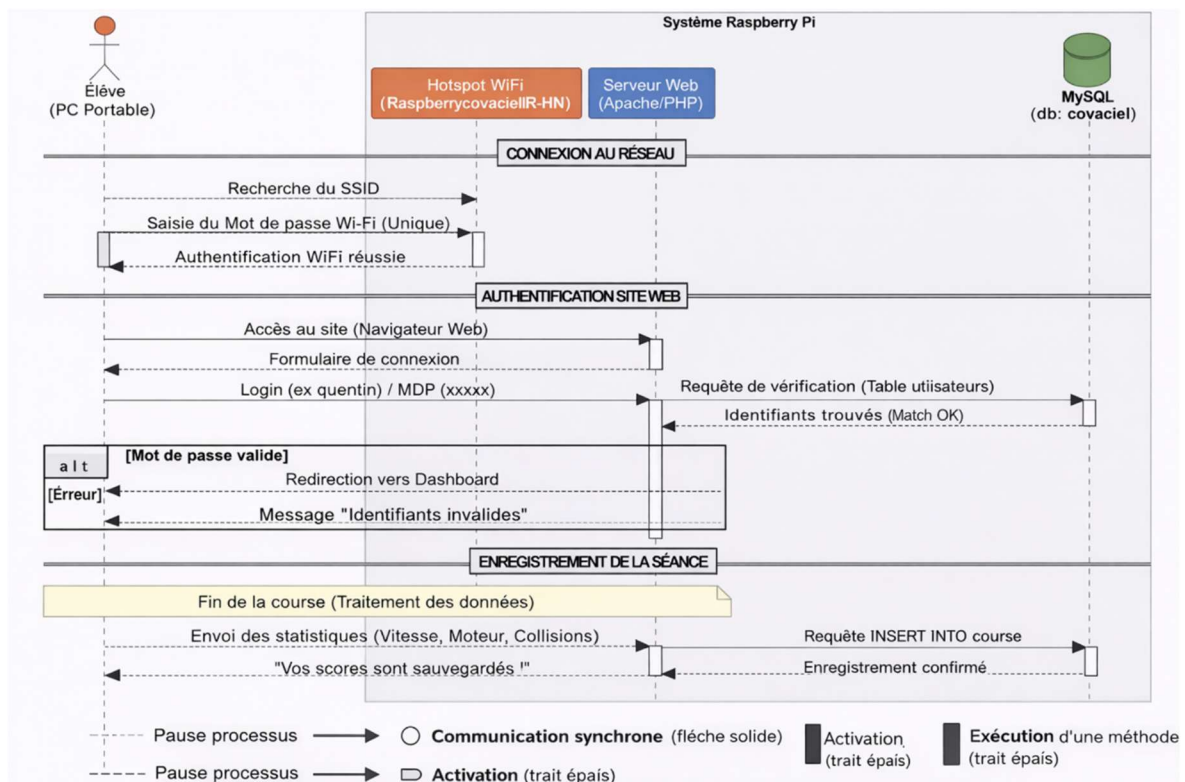


Figure 14 - Diagramme de séquence — Connexion, authentification et monitoring temps réel

Diagramme de cas d'utilisation

Le diagramme UC identifie trois acteurs et leurs interactions :

Acteur	Cas d'utilisation
Visiteur (non authentifié)	Consulter l'accueil, la page équipe, la page lycée, les statistiques publiques des courses
Administrateur (authentifié)	Se connecter, accéder au dashboard temps réel, consulter les données capteurs, gérer les utilisateurs, se déconnecter
Système voiture	Envoyer les données capteurs en temps réel via WebSocket (événements telemetry, zone_update, obstacle_update)

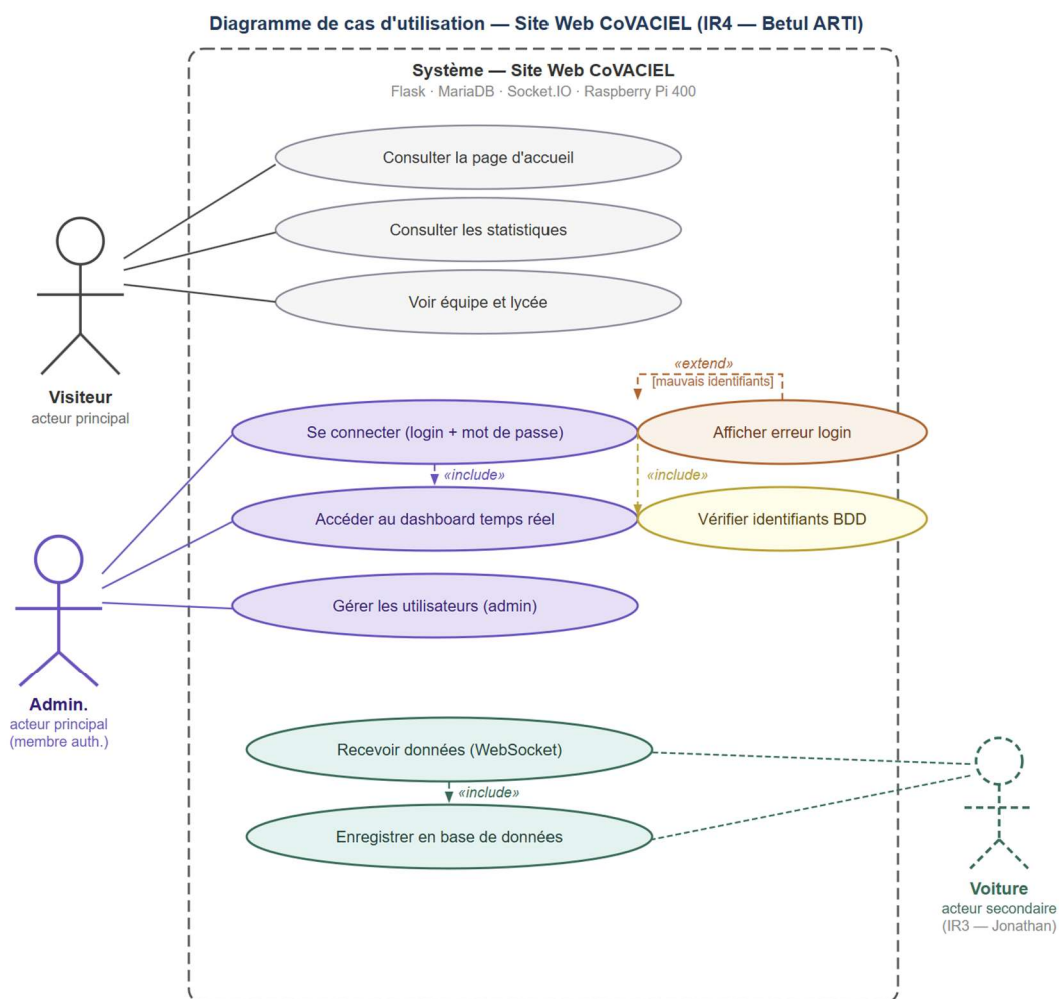


Figure 15 - Diagramme de cas d'utilisation — Acteurs : Visiteur, Administrateur, Système voiture

8.5 Réalisation du site web

Structure et navigation

Le site est composé de cinq pages reliées par un bandeau de navigation commun. Le design est inspiré de l'univers de la Formule 1 : fond sombre violet, couleurs cyan, animations sur les lignes du classement.



Figure 16 - Page d'accueil (index.html) — Présentation du projet CoVACIEL

Dashboard temps réel — WebSocket / Socket.IO

Le dashboard utilise Socket.IO pour créer un canal de communication permanent entre le serveur et le navigateur. Contrairement au web classique qui nécessite un rafraîchissement, les données sont poussées instantanément dès leur réception depuis la voiture.

Fonctionnement WebSocket

- Dès qu'un utilisateur ouvre le dashboard, le serveur émet immédiatement la dernière mesure connue (emit).
- Tant que la session reste ouverte, chaque nouvelle donnée capteur est broadcastée à tous les clients connectés.
- Badge de connexion : vert = connecté, rouge = déconnecté, jaune clignotant = reconnexion.
- Événements traités : telemetry, zone_update, obstacle_update.

Organisation 3 colonnes

- Colonne gauche (FRONT – Quentin) : zone de détection active (zones A→D LiDAR), distance avant minimale, nombre d'obstacles détectés.
- Colonne centrale : vue schématique de la voiture, badge Socket.IO.
- Colonne droite (Traction + Back) : vitesse m/s, RPM moteur, distance parcourue, distance arrière SRF10, état marche arrière.

```

# WEBSOCKET (Communication en temps réel)

# Quand un utilisateur ouvre le Dashboard, cette fonction se déclenche automatiquement
@socketio.on('connect')
def ws_connect():
    # Affiche dans la console du serveur l'identifiant unique de la personne qui vient de se connecter
    print(f'[WS] connecté : {request.sid}')

    # On prépare un accès à la base de données pour donner les dernières infos connues
    cur = mysql.connection.cursor()

    # On va chercher la TOUTE DERNIÈRE ligne enregistrée dans la table télémétrie (le dernier état de la voiture)
    cur.execute("SELECT * FROM telemetrie ORDER BY id DESC LIMIT 1")
    row = cur.fetchone() # On récupère cette ligne
    cur.close() # On ferme la connexion à la base de données

    # Si on a trouvé une donnée (si la voiture a déjà roulé au moins une fois)
    if row:
        # On vérifie si la date est au format Python (datetime)
        if isinstance(row.get('timestamp'), (date, datetime)):
            # On transforme la date en texte (format ISO) pour qu'elle soit lisible par le site web
            row['timestamp'] = row['timestamp'].isoformat()

        # ON ENVOIE la donnée au navigateur de l'utilisateur qui vient de se connecter
        # Comme ça, dès qu'il arrive, son écran n'est pas vide : il voit le dernier état.
        emit('telemetry', row)

# Quand l'utilisateur ferme l'onglet du site ou perd la connexion
@socketio.on('disconnect')
def ws_disconnect():
    # On affiche dans la console que la personne est partie
    print(f'[WS] déconnecté : {request.sid}')

```

Figure 17 - Dashboard temps réel (dashboardprv.html) — Vue complète 3 colonnes

Page statistiques

La page statistiques affiche les données historiques des courses sous forme de graphiques Chart.js alimentés dynamiquement depuis MySQL via Flask.

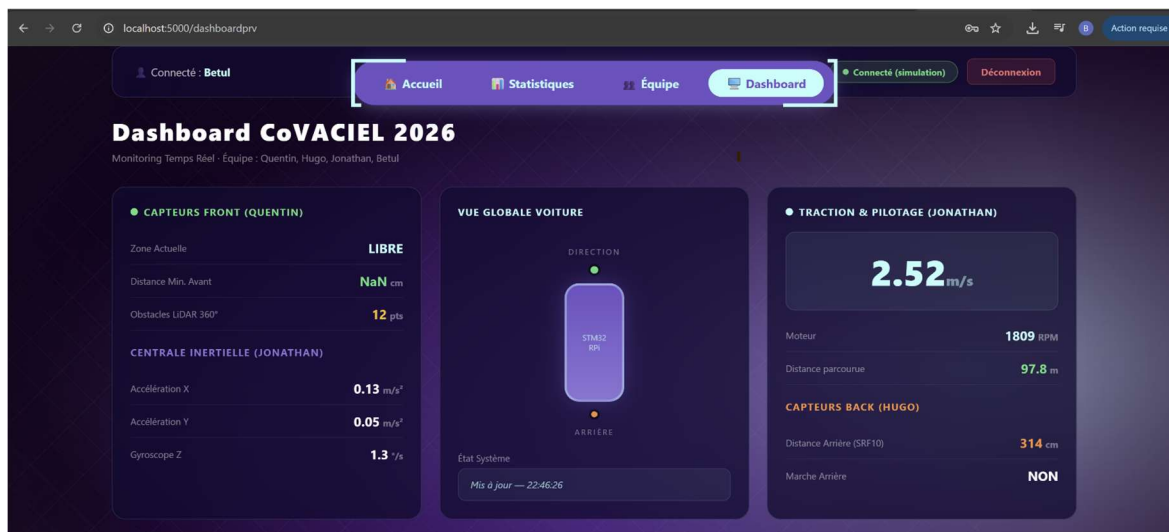


Figure 18 - Page statistiques — Graphiques Chart.js alimentés depuis MySQL

8.6 Hébergement sur Raspberry Pi 400

Environnement virtuel Python (venv)

Raspberry Pi OS interdit l'installation directe de paquets Python (protection externally-managed) pour éviter de dérégler le système. La solution est de créer un environnement virtuel isolé (venv) dans lequel Flask-SocketIO et les autres dépendances peuvent être installés librement.

Procédure de création du venv

- `python3 -m venv venv` # Crée le dossier venv/
- `source venv/bin/activate` # Active l'environnement
- `pip install flask flask-socketio flask-mysql-connector` # Installe les dépendances
- `python app.py` # Lance le serveur dans l'environnement isolé

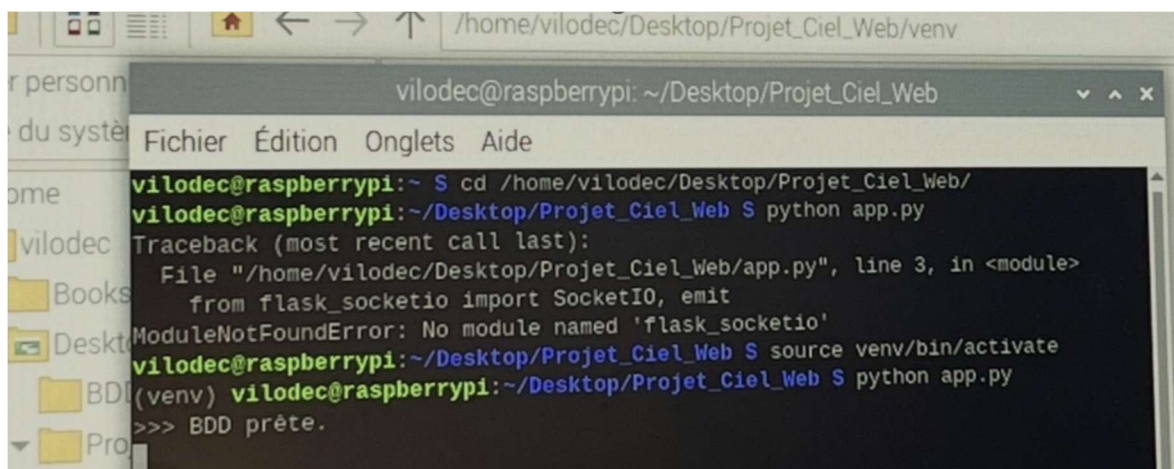


Figure 19 - Terminal Raspberry Pi — Erreur 'No module named flask_socketio' puis lancement correct avec venv

Installation de la pile LAMP

Après validation en local avec UwAmp, l'application a été déployée sur la Raspberry Pi via l'installation du stack LAMP (Linux, Apache2, MySQL/MariaDB, PHP). Les quatre commandes clés :

Installation du stack LAMP

- `sudo apt update && sudo apt upgrade -y` # Mise à jour du système
- `sudo apt install apache2 php libapache2-mod-php php-mysql mariadb-server phpmyadmin -y`
- `sudo systemctl enable apache2 mariadb` # Activation au démarrage
- `sudo mysql_secure_installation` # Sécurisation initiale de la base de données

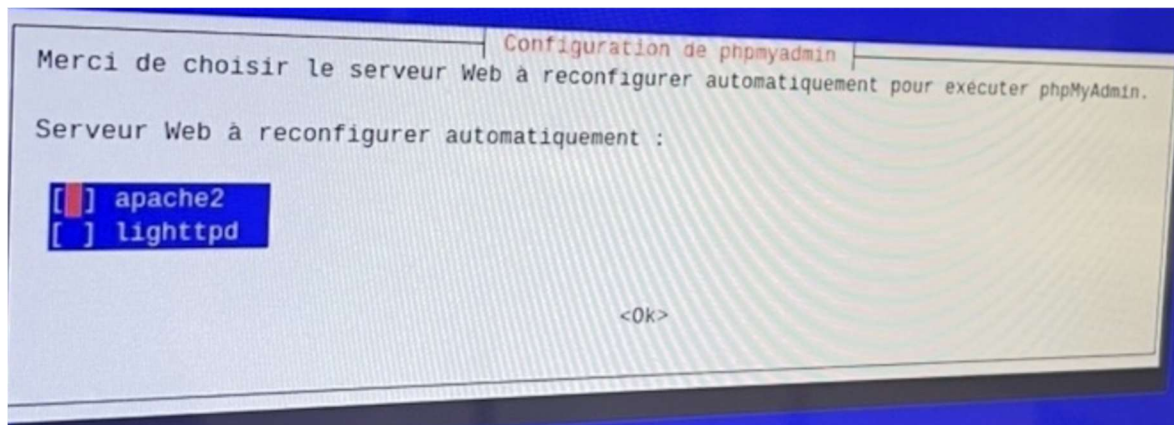


Figure 20 - Installation de la pile LAMP — Sélection du serveur web Apache2

8.7 Configuration réseau et sécurisation

Création du hotspot Wi-Fi

Sur le circuit de course, la Raspberry Pi 400 ne peut pas se connecter à la box internet du lycée. Elle crée son propre réseau Wi-Fi privé via NetworkManager, permettant aux téléphones des membres du jury et des spectateurs de se connecter.

Paramètre réseau	Valeur
Nom du réseau (SSID)	RaspberryCovaciellR-HN
Protocole de sécurité	WPA3 (chiffrement le plus récent — 2018)
IP Raspberry Pi (passerelle)	10.42.0.1
Plage DHCP	10.42.0.2 à 10.42.0.254 (attribution automatique)
URL site Flask	http://10.42.0.1:5000

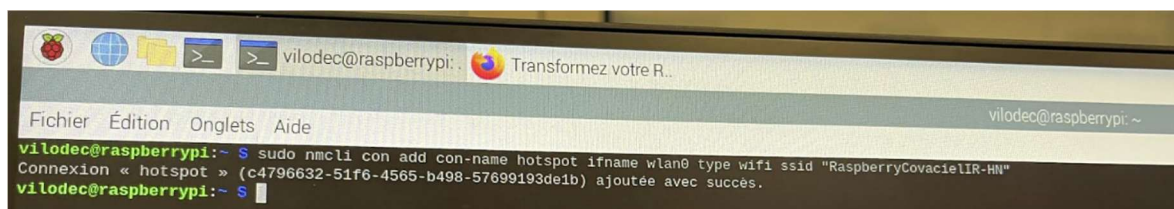


Figure 21 - Accès confirmé au site CoVACIEL via le hotspot Raspberry Pi

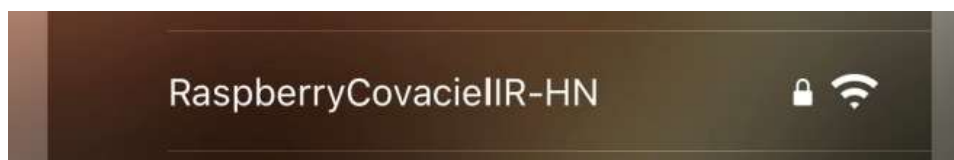


Figure 22 - Hotspot RaspberryCovaciellR-HN visible dans la liste Wi-Fi depuis un iPhone

Sécurisation par pare-feu (UFW)

Le pare-feu UFW est configuré pour n'autoriser que les ports strictement nécessaires :

Configuration UFW

```
- sudo ufw allow 80/tcp    # HTTP (Dashboard et API)
- sudo ufw allow 22/tcp    # SSH (Administration à distance)
- sudo ufw enable         # Activation du pare-feu
```

Protocole WPA3

WPA3 est le protocole de chiffrement Wi-Fi le plus récent (2018). Toutes les données de la voiture (vitesse, obstacles, direction) transmises via WebSocket sont ainsi protégées contre l'interception ou la modification pendant la course.



Figure 23 - Comparatif des protocoles Wi-Fi — WPA3 retenu pour le projet CoVACIEL

Sécurité applicative

La sécurité applicative repose sur quatre mécanismes Flask :

- Authentification par session Flask : les routes /dashboard, /statistiques et /admin/users vérifient session.get('logged_in') — toute requête non authentifiée est redirigée vers le login.
- Hachage SHA-256 : les mots de passe ne sont jamais stockés en clair dans la base de données.
- Protection contre les mauvais identifiants : en cas d'erreur, Flask redirige vers erreur.html avec un message explicite.
- Déconnexion sécurisée : la route /logout efface la session complète (session.clear()) avant de rediriger vers l'accueil.

Procédure de lancement

Pour accéder au site depuis la Raspberry Pi ou depuis un PC connecté au même réseau hotspot :

```
Fichier  Édition  Onglets  Aide
vilodec@raspberrypi:~ $ python app.py
python: can't open file '/home/vilodec/app.py': [Errno 2] No such file or directory
vilodec@raspberrypi:~ $ cd /home/vilodec/Desktop/Projet_Ciel_Web/
vilodec@raspberrypi:~/Desktop/Projet_Ciel_Web $ python app.py
>>> Connexion BDD réussie et tables initialisées.
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.99.204:5000
Press CTRL+C to quit
* Restarting with stat
>>> Connexion BDD réussie et tables initialisées.
* Debugger is active!
* Debugger PIN: 107-843-475
127.0.0.1 - - [20/Mar/2026 19:11:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2026 19:11:43] "GET /favicon.ico HTTP/1.1" 404 -
```

Figure 24 - Procédure de lancement du site CoVACIEL sur Raspberry Pi

8.8 Extraits de code clés

Les extraits suivants présentent les fonctions essentielles du serveur Flask.

Lancement du serveur Flask

Le paramètre `host='0.0.0.0'` est crucial : il permet à n'importe quel appareil connecté au hotspot (pas seulement localhost) d'accéder au site.

```
471
472 # LANCEMENT
473 if __name__ == '__main__': # Si on lance ce fichier directement (pas importé)
474     with app.app_context(): # Donne l'accès aux configurations de l'application Flask
475         try: # Teste le bloc de code qui suit pour voir s'il y a des erreurs
476             init_db() # Lance la création ou la connexion à la base de données
477         except Exception as e: # Si une erreur survient, on l'attrape ici
478             print(f"\n✗ Connexion MySQL impossible : {e}") # Affiche le message d'erreur
479             print(" → Vérifie MYSQL_HOST / MYSQL_USER / MYSQL_PASSWORD / MYSQL_DB\n") # Rappelle les variables à vérifier
480
481 # Démarre le serveur web avec SocketIO (0.0.0.0 = accessible partout, port 5000, debug actif)
482 socketio.run(app, host='0.0.0.0', port=5000, debug=True)
483
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS
* Restarting with stat
✓ Base de données prête
* Debugger is active!
* Debugger PIN: 142-053-471
(23004) wsgi starting up on http://0.0.0.0:5000
```

Figure 25 - Lancement du serveur Flask avec Socket.IO — `host='0.0.0.0'`

Gardien de sécurité — Vérification de session

Ce décorateur vérifie en arrière-plan si l'utilisateur est connecté avant chaque route protégée. Si ce n'est pas le cas, il bloque l'accès avec une redirection vers le login.

```
@app.route('/api/me', methods=['GET']) # Crée une route URL secrète '/api/me' accessible uniquement en Lecture (GET)
def api_me(): # Démarre la fonction Python
    """Retourne l'utilisateur connecté...""" # Phrase d'explication du code (docstring)
    if 'user' not in session: # VERIFICATION : Si le nom de l'utilisateur n'est PAS en mémoire (session)
        return jsonify({'error': 'non connecté'}), 401 # ALERTE : Renvoie une erreur texte et le code de sécurité 401 (Accès refusé)
    return jsonify(session['user']) # SUCCÈS : Si connecté, renvoie les infos de l'utilisateur (nom, rôle) au format JSON
```

Figure 26 - Gardien de sécurité Flask — Vérification de session avant accès au dashboard

Hachage SHA-256 et authentification

Le mot de passe saisi est haché avant d'être comparé à celui stocké en base. Si les deux hashes correspondent, Flask crée une session côté serveur et l'utilisateur est redirigé vers le dashboard.

```
# On crée une fonction pour transformer Le mot de passe en code secret
def hachage_mot_de_passe(mdp_clair):
    # 1. On transforme Le texte en format binaire (compréhensible par l'algo)
    donnees_binaires = mdp_clair.encode()

    # 2. On applique L'algorithme de sécurité SHA-256
    code_hache = hashlib.sha256(donnees_binaires)

    # 3. On retourne Le résultat sous forme de texte (hexadécimal) pour la base de données
    return code_hache.hexdigest()
```

Figure 27 - Hachage SHA-256 — Fonction de chiffrement du mot de passe

```
@app.route('/equipe', methods=['GET', 'POST'])
def equipe():
    cur = mysql.connection.cursor()

    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        cur.execute("SELECT * FROM utilisateurs WHERE username=%s AND password=%s", (username, hash_pw(password)))
        user = cur.fetchone()

        if user:
            session['user'] = user
            return redirect(url_for('dashboard'))
        else:
            error = 'Identifiants incorrects'
```

Figure 28 - Authentification Flask — Comparaison hash et création de session

Push WebSocket temps réel

Dès qu'un utilisateur ouvre le dashboard, le serveur lui envoie instantanément la dernière mesure de la voiture. Ensuite, chaque nouvelle donnée reçue est broadcastée à tous les clients connectés simultanément.

```
@socketio.on('connect')
def ws_connect():
    cur = mysql.connection.cursor()

    cur.execute("SELECT * FROM telemetrie ORDER BY id DESC LIMIT 1")
    row = cur.fetchone()

    emit('telemetry', row)
```

Figure 29 - Push WebSocket temps réel vers tous les dashboards connectés

Route statistiques

Ce code s'exécute lors de la visite de la page des statistiques. Il calcule automatiquement le nombre total de courses et la vitesse moyenne depuis la base de données, puis les transmet au template Jinja2.

```
@app.route('/statistiques')
def statistiques():
    cur = mysql.connection.cursor()

    cur.execute("SELECT COUNT(*) AS total FROM courses")
    total = cur.fetchone()['total'] or 0

    cur.execute("SELECT ROUND(AVG(vitesse_moy),2) AS moy FROM courses")
    moy = cur.fetchone()['moy'] or 0

    cur.execute("SELECT * FROM courses ORDER BY date_course DESC")
    courses = cur.fetchall()

    return render_template('statistiques.html', courses=courses, ...)

# Crée la page des statistiques
# Démarre la fonction
# Ouvre le curseur pour la base de données
# Compte le nombre total de courses
# Récupère le total (met 0 si vide)
# Calcule la vitesse moyenne
# Récupère la moyenne (met 0 si vide)
# Récupère toutes les courses (de la plus récente à la plus ancienne)
# Enregistre toute la liste
# Envoie les données à la page HTML
```

Figure 30 - Route /statistiques Flask — Requêtes MySQL et rendu du template

8.9 Problèmes rencontrés et solutions

Problème rencontré	Cause identifiée et solution adoptée
ModuleNotFoundError: flask_socketio	Protection externally-managed sur Raspberry Pi OS. → Création d'un environnement virtuel Python isolé (venv).
Erreur SQL 1045 Access Denied	MariaDB utilise l'authentification socket Unix pour root. → ALTER USER avec mysql_native_password.
IP 169.254.x.x — pas de DHCP	Serveur DHCP hotspot non actif. → Mode « Partagé avec d'autres ordinateurs » dans NetworkManager. IP fixée à 10.42.0.1.
Latence 20s / affichage cassé	CDN inaccessibles sans internet. → Embarquement local de toutes les librairies JS/CSS (Chart.js, Socket.IO, Bootstrap).
socketio.run() missing 'app'	Argument app Flask manquant en 1er paramètre. → socketio.run(app, ...).
Erreur Invalid async_mode (gevent)	gevent absent du venv Raspberry Pi. → async_mode='threading' (natif Python).
Données absentes du tableau statistiques	Noms de colonnes différents entre Flask et MySQL. → Harmonisation des noms dans app.py et les templates.
Lenteur sur Raspberry Pi	Images trop lourdes + debug=True actif. → Images allégées + debug=False en production.
Hotspot instable	Une seule puce Wi-Fi : hotspot et client internet impossibles simultanément. Contrainte matérielle connue.

8.10 Bilan personnel

Ce projet m'a permis de développer des compétences concrètes dans plusieurs domaines complémentaires :

Domaine	Compétences acquises
Python + Flask	API REST, routes HTTP, sessions, connexion MySQL, templates Jinja2, SocketIO
HTML5 / CSS3 / JavaScript	Pages HTML, thème aurora CSS, graphiques Chart.js, gestion Socket.IO client
SQL / MariaDB	Modélisation 3 tables, hachage SHA-256, SELECT/INSERT, phpMyAdmin
Réseau & Linux	Hotspot WPA3 (NetworkManager), DHCP, SSH, commandes bash (apt, systemctl, ufw)
Gestion de projet	GanttProject, Drive partagé, GitHub, coordination avec IR1, IR2, IR3

Points forts : déployer un serveur web complet sur un Raspberry Pi est la réalisation la plus concrète de ma scolarité en BTS CIEL. La résolution des erreurs en conditions réelles (hotspot, MariaDB, venv) a développé mon autonomie technique et ma capacité à diagnostiquer des problèmes dans un environnement embarqué.

Esprit d'équipe : ce projet CoVACIEL 2026 a été une expérience enrichissante. Quand un bug bloquait l'avancement d'un membre, toute l'équipe se mobilisait pour trouver une solution. La coordination entre les quatre parties (pilotage, LiDAR, capteurs arrière, dashboard) a exigé une communication constante et rigoureuse.

En juin 2026, sur le circuit de Pessac à Bordeaux, quand la voiture partira en autonomie et que les données s'afficheront en direct sur le dashboard, ce sera la concrétisation de plusieurs mois de travail collectif — et la preuve qu'une équipe de quatre étudiants en BTS CIEL peut concevoir un système autonome complet.

9. Bilan et conclusion

9.1 Bilan des réalisations

Fonctionnalités réalisées et validées

- Acquisition LiDAR 180° avant avec le RPLIDAR A2M12, grille 7×3 stable à 50 Hz
- Pilotage autonome RACE_CENTER, angle et vitesse calculés en temps réel (cycle de 20 ms)
- Accélération progressive et freinage adaptatif selon six niveaux de taux distincts
- Détection arrière avec le SRF10, sécurité de la marche arrière opérationnelle
- Réception du signal de départ XBee, déclenchement du mode automatique
- Alimentation du servomoteur corrigée (BEC de l'ESC) : saccades supprimées
- Interface de monitoring courses sur le Raspberry Pi
- Cartes HAT et alimentation (réalisées par Emmanuel KEITH) intégrées et câblées
- Communication ESP32 vers Raspberry Pi : JSON atomique, 50 Hz, stable
- Surveillance automatique des communications (côté ESP32 et Raspberry Pi)
- Protection des accès concurrents validée sur 200 grilles aléatoires
- Dashboard web de télémétrie temps réel connecté au flux de données

9.2 Leçons clés du projet

Pragmatisme technologique : une solution fonctionnelle vaut mieux qu'une solution théoriquement optimale. La migration du STM32 vers l'ESP32 a débloqué le projet en une semaine.

Rigueur dans la lecture des spécifications : le bug du bit de contrôle illustre l'importance de lire attentivement les fiches techniques avant toute implémentation.

Importance du découplage des alimentations : le problème de saccades du servo a démontré qu'en système embarqué multi-actionneurs, chaque actionneur doit disposer d'une alimentation dimensionnée pour son courant de pointe.

Tests systématiques et progressifs : la validation sur 200 grilles aléatoires puis par étapes sur piste a permis de certifier la robustesse du système de manière objective.

Suivi de version rigoureux : conserver les versions fonctionnelles a permis de diagnostiquer le bug LiDAR par comparaison directe du code, sans débogage à l'aveugle.

9.3 Conclusion

Ce projet nous a permis de concevoir, réaliser et valider un système complet de véhicule autonome dans le cadre d'une compétition nationale. Nous avons su faire face aux obstacles techniques rencontrés — migration du microcontrôleur, bug de décodage du protocole LiDAR, problème d'alimentation du servomoteur — en les résolvant grâce à une méthode rigoureuse, des tests progressifs et un travail d'équipe coordonné.

Nous abordons la compétition de juin 2026 avec un système entièrement intégré, testé et validé. Ce projet nous a apporté des compétences concrètes en systèmes embarqués, en protocoles de communication série, en intégration électronique et en développement logiciel appliqué à la robotique autonome.

Index des figures

Figure 1 - Planning prévisionnel du projet CoVACIEL (janvier à juin 2026).....	7
Figure 10 - Table courses dans phpMyAdmin — Données statistiques par course.....	36
Figure 11 - Table télémétrie dans phpMyAdmin — Mesures capteurs en temps réel	37
Figure 12 - Diagramme de séquence — Connexion, authentification et monitoring temps réel	38
Figure 13 - Diagramme de cas d'utilisation — Acteurs : Visiteur, Administrateur, Système voiture.....	39
Figure 14 - Page d'accueil (index.html) — Présentation du projet CoVACIEL.....	40
Figure 15 - Dashboard temps réel (dashboardprv.html) — Vue complète 3 colonnes	41
Figure 16 - Page statistiques — Graphiques Chart.js alimentés depuis MySQL	41
Figure 17 - Terminal Raspberry Pi — Erreur 'No module named flask_socketio' puis lancement correct avec venv	42
Figure 18 - Installation de la pile LAMP — Sélection du serveur web Apache2	43
Figure 19 - Accès confirmé au site CoVACIEL via le hotspot Raspberry Pi	43
Figure 2 - Planning de suivi hebdomadaire du projet CoVACIEL (janvier à mai 2026).....	7
Figure 20 - Hotspot RaspberryCovaciellR-HN visible dans la liste Wi-Fi depuis un iPhone ..	43
Figure 21 - Comparatif des protocoles Wi-Fi — WPA3 retenu pour le projet CoVACIEL	44
Figure 22 - Procédure de lancement du site CoVACIEL sur Raspberry Pi.....	45
Figure 23 - Lancement du serveur Flask avec Socket.IO — host='0.0.0.0'	45
Figure 24 - Gardien de sécurité Flask — Vérification de session avant accès au dashboard	45
Figure 25 - Hachage SHA-256 — Fonction de chiffrement du mot de passe	46
Figure 26 - Authentification Flask — Comparaison hash et création de session	46
Figure 27 - Push WebSocket temps réel vers tous les dashboards connectés	46
Figure 28 - Route /statistiques Flask — Requêtes MySQL et rendu du template.....	47
Figure 3 - Diagramme de bloc SysML — Architecture matérielle du système CoVACIEL.....	8
Figure 4 - Diagramme de cas d'utilisation SysML — CoVACIEL.....	10
Figure 4 – Photo voiture CoVACIEL avant 3/4.....	9
Figure 5 - Diagramme d'exigences SysML — Hiérarchie des exigences CoVACIEL	10
Figure 5 – Photo voiture CoVACIEL arrière 3/4	9
Figure 6 - Capture d'écran du logiciel Frame Grabber	13
Figure 7 - Logo UwAmp — Environnement de développement local (Apache, MySQL, PHP)	35
Figure 8 - MLD de la base de données covaciel — Tables utilisateurs, courses et télémétrie	35
Figure 9 - Table utilisateurs dans phpMyAdmin — Structure et champs.....	36